IE University

School of Science & Technology



Domain knowledge or surface writing? A Cross-layer Analysis of Sparse Autoencoders on specialised Language Models

Interpretable feature extraction on Llama 3.2 1B Instruct on biomedical text

Baptiste Etroy

Bachelor of Computer Science & Artificial Intelligence

Supervised by:

Luciano Dyballa

Connection Science, MIT, Cambridge, USA

Robotics & AI Lab, School of Science & Technology, IE University, Spain

10th April 2026

Abstract

Sparse autoencoders (SAEs) have become the primary tool for extracting interpretable features from large language models and understanding behaviour. Nearly all published work applies them to general models trained on web text. Therefore if we use these methods on models specific to a domain, say law, do they actually encode genuine domain knowledge or just the surface conventions of domain specific writing? In areas such as law, science, and medicine, exactness is essential. This thesis develops a mechanistic interpretability pipeline centred on BatchTopK sparse autoencoders and applies it to Meta's Llama 3.2 1B Instruct. The SAEs are trained on residual stream activations collected while the model processes approximately 46 million tokens of biomedical text drawn from three sources (MedMCQA, PubMed QA and PubMed Abstracts).

A cross layer analysis of BatchTopK SAEs trained independently on layers 4, 8, and 12 of the model's residual stream across 8 epochs was executed. An interactive web application was developed to explore features and control model behaviour. At layer 8, the largest feature cluster corresponds to research methodology conventions, while genuine biomedical content accounts for a smaller proportion. But at layer 12, this balance shifts: domain-knowledge features increase in proportion and methodology features decline. Per-feature source attribution confirms the pattern independently of labelling, showing that methodology features draw activations from PubMed abstract prose while clinical features draw from question-answer formatted medical data.

The central finding is that the transition from language-about-medicine to medicine-itself is a layer-depth phenomenon. This has direct implications for interpretability audits of domain-specialised systems, where layer selection determines whether an analysis captures genuine domain representations or surface writing conventions. While demonstrated on biomedical text, the pipeline and its contributions are domain-agnostic.

Keywords: mechanistic interpretability, sparse autoencoders, feature extraction, monosemanticity, domain specialisation, cross-layer

Contents

1 Introduction **6**

2 Literature Review **9**

2.1 Feature Extraction 9

2.2 Automated Interpretation Pipelines 10

2.3 Behaviour Control 11

2.4 Attribution, Causality, and Evaluation 12

2.5 Key Findings 13

2.6 Applications and Limitations 15

2.7 Synthesis 15

3 Methodology **18**

3.1 Pipeline Overview 18

3.2 Model Selection 18

3.3 Biomedical Dataset Construction and Activation Collection 19

3.3.1 Data Sources 19

3.3.2 Chat Template Formatting 19

3.3.3 Tokenisation and Activation Extraction 20

3.4 BatchTopK Sparse Autoencoder 20

3.4.1 Architecture 20

3.4.2 BatchTopK Activation 21

3.4.3 Initialisation 21

3.4.4 Training Objective 22

3.4.5 Optimiser and Schedule 22

3.4.6 Training Diagnostics 22

3.5 Feature Analysis Pipeline 23

3.5.1 Pass 1: Basic Feature Statistics 23

3.5.2 Pass 2: MaxAct Input-Centric Interpretation 23

3.5.3 Pass 3: VocabProj Output-Centric Interpretation 24
3.5.4 Pass 4: TokenChange Causal Interpretation 25
3.6 Post-Training Evaluation Metrics 25
3.7 Monosemanticity Quantification and Feature Quality Scoring 26
3.7.1 Shannon Entropy as Monosemanticity Proxy 26
3.7.2 Multi-Component Quality Score 26
3.8 Feature Labelling Pipeline 27
3.8.1 Stage 1: Heuristic Classification 27
3.8.2 Stage 2: LLM Semantic Labelling 27
3.8.3 Semantic Categories 28
3.9 Per-Feature Source Attribution 28
3.10 Cross-Layer Analysis 29
3.11 Behaviour Control Interface 30
3.11.1 Chat and Explore 30
3.11.2 Feature Steering 30
3.11.3 Feature Browser 31

4 Experiments **32**
4.1 Experimental Design Overview 32
4.2 Dataset Configuration 32
4.3 SAE Architecture and Training 33
4.4 Feature Labelling Pipeline 34
4.5 Cross-Layer Analysis 36

5 Results **37**
5.1 SAE Training: L1 Regularisation vs. TopK Activation 37
5.2 Feature Labelling Results 39
5.3 Main Empirical Finding: SAEs Learn the Language of Medical Research, Not Medical Knowledge 39

5.4 Cross-Layer Analysis: Feature Composition and Depth-Dependent Semantic Emergence . . . 41
5.5 Qualitative Feature Analysis and Behaviour Control . . . 44
5.5.1 Feature Case Studies . . . 45
5.5.2 VocabProj Reliability . . . 45
5.5.3 Behaviour Control via Feature Steering . . . 46

6 Discussion **47**
6.1 What SAEs Learn in Specialised Language Models . . . 47
6.2 Cross-Layer Feature Composition . . . 47
6.3 Training Dynamics: Dead Features and the L1–TopK Trade-off . . . 48
6.4 Limitations . . . 49

7 Future Work **51**
7.1 Resolving the Mid-Layer Dead Feature Problem . . . 51
7.2 Systematic TokenChange Evaluation . . . 51
7.3 Scaling to Larger Models . . . 52
7.4 Additional Hook Points . . . 52
7.5 Domain Transfer . . . 52
7.6 Human Expert Validation . . . 53

8 Conclusion **54**
8.1 Contributions . . . 55
8.2 Implications for Practice . . . 55

A Hyperparameter Configuration **56**

B Quality Score Details **57**

C Semantic Category Taxonomy **58**

D Selected Feature Examples **59**

E Layer 4 and Layer 8 Category Distributions **62**

F Behaviour Control UI Screenshots **62**

1 Introduction

Large language models are currently the forefront of the technological world. As they continue to grow and get deployed, we increasingly use and trust them. But there are domains where errors can carry real consequences: education, healthcare, law. A model that misidentifies a symptom or hallucinates a legal precedent will produce a plausible answer, and a plausible wrong answer is dangerous. The standard response is to treat the model as a black box and evaluate it on benchmarks, but benchmarks measure what a model gets right on average. They do not explain why a model produces this particular output, or which input drove its answer. Benchmarks do not explain which internal representations it used, or whether those representations actually encode genuine knowledge of the domain as opposed to surface-level patterns that happen to correlate with correct answers on a test set. By looking inside the model, we aim to answer these questions. The bottom line is that transformer models develop internal features that correspond to interpretable concepts. These features are key to understanding model behaviour, once extracted we can label and study them.

Sparse autoencoders (SAEs) have become the primary extraction tool. They decompose a model's residual stream activations into a sum of interpretable components (which we call features), each ideally corresponding to a single semantic concept, which we call "monosemanticity". For example, anthropic's work on Claude managed to identify very specific features such as the Golden Gate Bridge and the Rosetta Stone (Templeton et al., 2024). More work has shown that SAE dictionaries are able to capture millions of features across different layers (Paulo et al., 2025). Most of this work has however been conducted on general models trained on web text and general internet corpora. This raises some questions, so we ask ourselves: what do we find inside a model trained on a specific domain? Do the extracted features reflect genuine domain expertise or do they simply capture the domain specific writing? A model that learned the linguistic habits of medical data will look very different, internally, from one that learned the medical data itself. An interpretability audit that cannot tell these apart is of limited use.

This capstone project develops a mechanistic interpretability pipeline built around a Batch-

TopK SAE, applied to the open-weight Meta Llama 3.2 1B Instruct. The SAE is trained on residual stream activations collected while the model processes approximately 46 million tokens of biomedical text from three sources (MedMCQA, PubMed QA and PubMed Abstracts). The pipeline combines several methodological choices, some adapted from existing work and some new. Once features are extracted we assess their quality before labelling, to prioritise features the most likely monosemantic. The quality score is assessed using a multi-component heuristic that incorporates Shannon entropy over the MaxAct token distribution as a monosemanticity proxy, alongside activation frequency, token diversity and coverage. The surviving features are ranked based on score and pass through labelling in two stages: a rule based heuristic, to classify all positional, syntactic and digit features automatically, then the remaining are labelled with an LLM (GPT 4o-mini) using the MaxAct examples, mean-centred VocabProj tokens and source breakdown. The latter is, to my knowledge, a novel contribution: per-feature dataset attribution. For every feature extracted, the pipeline records which training corpus each MaxAct token originated from, producing a breakdown of training origin for each individual feature. This helps us determine whether an activation signal comes from medical Q&A data or research paper writing style, independently of the semantic label it receives.

The central empirical finding from this capstone is that SAEs trained on this medical model mainly learn the language of medical research rather than knowledge itself. At the mid-layer

residual stream (layer 8), the largest cluster of labelled features corresponds to research methodology: hedged claim language, statistical reporting phrases, methods-section discourse. Genuine domain content (anatomy, pharmacology, biology) is considerably scarce. The source attribution data confirms it: methodology features come from PubMed abstracts, while clinical features draw more heavily from the question-answer formatted MedMCQA corpus. However, the conducted cross-layer analysis reveals that this isn't the whole story. At layer 12, we see a substantial rise in domain-knowledge features and a drop of methodology features. The transition from language-about-medicine to medicine-itself is a depth-dependent phenomenon, that occurs between layers 8 and 12 of our model. An interpretability audit conducted only at mid-layer depth may actually underestimate a model's domain-specific representations.

While the project is centred around a biomedical domain, neither the pipeline nor the find-

ings are inherently domain-specific. SAEs are likely to learn writing style before actual content in any domain with a distinctive way of writing, from legal documents to scientific papers to code.

This thesis is organised as follows: Section 2 on literature reviewed, Section 3 describes the pipeline methodology, Section 4 explains the experiments and decisions undertaken, Section 5 presents the results, Section 6 the findings and limitations, Section 7 on future work and Section 8 concludes.

2 Literature Review

2.1 Feature Extraction

This literature review examines different studies published between 2023 and 2025, all focusing on different mechanistic interpretability approaches. The purpose of this review is to identify effective methods and approaches, as basis for designing new experiments and testing strategies in this research.

These studies primarily focus on open weight models (Marks et al., 2025) whose sizes range up to 70 billion parameters (Dunefsky et al., 2024), covering multiple model architectures such as GPT-2, Pythia, Gemma and LLaMA families. Notably, all reviewed works apply their methods to models trained on general internet corpora such as The Pile, OpenWebText, or RedPajama.

SAEs are the most used technique for extracting interpretable features from LLMs in our studies. SAEs are an unsupervised technique for decomposing the activations of a neural network into a sum of interpretable components (which we call features) (Shu et al., 2025). The basic architecture has an encoder, decoder, and a loss function with sparsity regularisation, in which sparsity is encouraged by an L1 term that promotes sparse feature activations (Marks et al., 2025). Cunningham et al. (2023) and Bricken et al. (2023) established in their work that SAEs extract more interpretable and monosemantic features than individual neurons.

Newer architectural innovations have addressed the limitations of L1-regularised SAEs. Gao et al. (2024) introduced TopK SAEs, which enforce exact sparsity by selecting the top K active features per forward pass rather than relying on an L1 penalty. They also proposed an auxiliary loss, giving a gradient signal to dead features, making sure autoencoder size and sparsity scales properly. Bussmann et al. (2024) followed those steps and introduced Batch-TopK. Now the sparsity constraint is at the batch-level, where the top $B \times K$ activations are selected across the entire batch rather than the top K per sample. With this method the model can use more features for complex inputs and less for simpler ones, keeping sparsity low but enabling a good reconstruction. Rajamanoharan et al. (2024) proposed to eliminate dead features

completely by using JumpReLU SAEs. It replaces the activation function with a learnable per-feature threshold. Each of these variants, along with ProLU SAEs, Switch SAEs, Layer Group SAEs, and Feature Aligned SAEs (Shu et al., 2025), each addresses a particular problem including efficiency, feature alignment between layers, and higher reconstruction fidelity. Various experiments train SAEs on each layer's attention output, MLP output, and residual stream, to then have broader set of features (Laptev et al., 2025). Templeton et al. (2024) experimented on production models, managing to extract up to 34 million features from Claude 3 Sonnet's mid-layer residual stream. They demonstrated that features are very abstract, multilingual, and steerable.

A big challenge that comes with training SAEs is that it is very computationally expensive and data-intensive. It needs a substantial amount of resource demands due to overcomplete dictionaries and layer-specific training requirements (Shu et al., 2025). To prevent "dead" neurons, features need to be reinitialised, which adds more training complexity (Marks et al., 2025). Despite all these costs, SAEs are still simpler to train than the original models, they require less compute, making them practical for interpretability research (Cunningham et al., 2023).

Transcoders are a different approach to feature extraction. They are designed specifically for the feed forward components (MLP sublayers), they achieve sparse activation while maintaining faithfulness to the original computation (Dunefsky et al., 2024). They perform at least on par with SAEs in terms of sparsity, faithfulness, and human-interpretability (Dunefsky et al., 2024). A second different method, SelfIE, leverages the LLM's own decoding capability to produce natural language descriptions of hidden embeddings, enabling open-world interpretation without requiring additional training or data collection (Chen et al., 2024).

2.2 Automated Interpretation Pipelines

Once feature clusters are extracted they must be labelled, and generating human understandable labels without manual intervention requires a precise automated method. The approach we see the most uses large language models as explainer models to generate natural language descriptions of SAE extracted features (Paulo et al., 2025). Table 1 summarises the main interpretation

methods.

Method	Approach	Computational Cost	Primary Advantage	Limitation
MaxAct	Shows activating examples to explainer model	High	Captures what triggers features	Fails to capture causal effects on outputs
VocabProj	Projects features onto vocabulary space	Low	Efficient, captures output effects	Depends on model vocabulary
TokenChange	Analyses output changes when feature amplified	Low	Captures causal effects efficiently	Depends on model vocabulary
Ensemble	Combines input- and output-centric methods	Medium	Best performance on both evaluations	Higher complexity

Table 1: Comparison of Feature Interpretation Methods

In the papers reviewed, interpretation pipeline architecture generally carries through a number of steps. Latent activations are first gathered from SAEs on large datasets (Paulo et al., 2025). The pipeline subsequently samples the examples on the various quantiles to represent a variety of activating situations (Paulo et al., 2025). An explainer model is used to produce natural language descriptions that focus on activating tokens and their strength (Paulo et al., 2025). These interpretations are evaluated on the basis of various scoring methods such as simulation, detection, fuzzing, surprisal and embedding scoring (Paulo et al., 2025). A critical innovation is intervention scoring, an evaluation of interpretability of feature interventions in terms of their counterfactual effect on model output (Paulo et al., 2025), especially discovering ”dead” features using output centered descriptions to identify activating inputs (Gur-Arieh et al., 2025).

For feature clustering, cosine similarity is used on decoder weights and pairs of context and token which have been vectorised (Laptev et al., 2025). The pipeline can operate without supervision, but human aid help to interpret results and verify (Marks et al., 2025).

2.3 Behaviour Control

Once we have extracted these interpretable features, they offer us control of the model behaviour. We can amplify features, suppress them, remove or implement them at different levels (Templeton et al., 2024).

Sparse Human-Interpretable Feature Trimming (SHIFT) visually inspects feature circuits to identify and eliminate task-irrelevant features (Marks et al., 2025). Backdoor Attribution

Vectors can boost the attack success rate to almost 100% on clean inputs or suppress it to almost 0% on triggered inputs with single-point intervention (Yu et al., 2025). In cross-layer feature mapping, flow graphs demonstrate the development of features between layers, offering multi-layer intervention which is proven more efficient than single layer (Laptev et al., 2025). SelfIE uses Supervised Control to edit individual layers and Reinforcement Control to erase harmful knowledge without requiring supervision targets (Chen et al., 2024). It reduces harmful responses by 84.66% and increases fairness in responses by 95% (Chen et al., 2024).

Despite all these advances, it still is hard to control behaviour. SHIFT works well only with high-quality SAEs (Marks et al., 2025) and models can 'self-repair' after interventions (Laptev et al., 2025).

2.4 Attribution, Causality, and Evaluation

To understand how internal features influence model outputs we can also use attribution analysis methods. Studies show tracing the influence through computational graphs, revealing a pathway from the inside to the output.

Sparse feature circuits are constructed using linear approximations to compute indirect effects between features, so we can identify and filter nodes and edges based on their root cause (Marks et al., 2025). Transcoders uses a weights-based analysis of MLP sublayers (Dunefsky et al., 2024). They calculate attributions between transcoder characteristics and decomposes them into input-dependent and input-invariant ones, which gives us general patterns of the circuit behaviour (Dunefsky et al., 2024). A greedy algorithm makes computational graphs where the attribution of a node is the sum of its incoming edges (Dunefsky et al., 2024). Flow graphs track feature evolutions across consecutive layers (Laptev et al., 2025). With data-free cosine similarity methods, the method follows the tracks of how a certain feature is maintained, modified, or initially introduced in each stage (Laptev et al., 2025). This approach reveals computational subnetworks by finding circuits where MLP and attention modules add or delete features (Laptev et al., 2025). Backdoor Attention Head Attribution (BAHA) uses the tripartite causal analysis model to identify particular attention heads which process the backdoor features (Yu et al., 2025). SelfIE decodes latent embeddings into sentences and maps feature influence

on downstream tokens (Chen et al., 2024). Ablation methods remove or modify features and look at output changes (Cunningham et al., 2023), trying to provide a perspective on viewing language models with sparse causal dependencies (Cunningham et al., 2023).

For interpretability quality, there are multiple different metrics such as mean squared error (MSE) or cosine similarity that measure how well SAEs preserve original activations (Shu et al., 2025). Cross-entropy loss, KL divergence, and delta LM loss evaluate shifts in probability distributions. We can assess sparsity through L0 norm measurements, latent firing frequency, and feature density statistics (Shu et al., 2025). Human evaluation remains important, feature interpretability is rated on 0-100 scales by crowdworkers (Marks et al., 2025). Input-faithfulness and output-faithfulness of features descriptions are rated by human annotators (Gur-Arieh et al., 2025). In studies, there is a measurement of correlation between automated scores and human judgments (Paulo et al., 2025), so that automated procedures are consistent with the human comprehension. The evaluation methodology is a trade off between detail and cost. Even with the top-k postprocessing of neurons that have been sparsified, SAEs always excel in interpretation over individual neurons (Gur-Arieh et al., 2025).

Recent work has also introduced feature-level distributional diagnostics. Heap et al. (2026) measured the Shannon entropy of the distribution of feature activations over token IDs as a proxy for feature "abstractness": low entropy indicates a feature that fires on a narrow set of specific tokens, while high entropy indicates a feature responding to many diverse tokens. They found that entropy increases with layer depth in trained models but remains flat in randomly initialised ones, suggesting that entropy captures a property of learned computation rather than architectural bias. This metric is relevant to feature quality assessment because it provides a scalar diagnostic for monosemanticity that requires no human annotation and can be computed programmatically at scale.

2.5 Key Findings

Experiments have shown that sparse autoencoders are successful in extracting more interpretable and monosemantic features as compared to other methods. SAEs de-intertwine superimposed features into explainable units and give insights about the processing of LLM (Shu

et al., 2025). The high-scale analysis proves that SAEs are easier to interpret than neurons and residual stream SAEs score higher than the ones trained on the output of the MLP (Paulo et al., 2025).

Output-centric models such as VocabProj and TokenChange are more causal in the effect of features on model outputs than input-centric models (Gur-Arieh et al., 2025). Input-output centric combinations of methods are better at describing features that can be evaluated well on both input and output analysis. These techniques work very well at finding previously ”dead” features. (Gur-Arieh et al., 2025)

Transcoders can be as sparse, faithful and human interpretable as SAEs (Dunefsky et al., 2024). The cross-layer feature flow analysis proves that multi-layer interventions are better in their performance than single-layer interventions. (Dunefsky et al., 2024)

Backdoor attribution studies prove great accuracy in controlling behaviour (Yu et al., 2025). Specific behaviours can be localised to small groups of attention heads and that they can be highly targeted in the interventions. (Yu et al., 2025)

SelfIEs' capacity to expose inner rationales processes such as ethical choice and judgment of harmful knowledge offers new information into LLM processes. (Chen et al., 2024)

Similarity analysis of features indicates that SAEs trained on adjacent layers of general-domain models produce closely similar feature dictionaries, suggesting structured internal mechanisms and the possibility of transfer learning among layers (Paulo et al., 2025). The implication of this finding is that feature representations may be partially stable to network depth in general-domain models, which could reduce the computational cost of training layer-specific SAEs. Whether this cross-layer similarity holds for domain-specialised models — where different layers may develop qualitatively different representations of domain content versus writing conventions — is an open question addressed by the present work.

The semantic analysis demonstrates that SAEs create dictionaries of concepts (Shu et al., 2025). They are able to learn huge amounts of concept vectors and fine-grained semantic concepts and manage model output in respect to toxicity, refusal and emotions (Shu et al., 2025).

2.6 Applications and Limitations

This research can be applied across diverse domains. Transparent and controllable models are becoming essential in today's world.

Safety and alignment comes first. We want to avoid the harmful output, rebuild the vectors to reject the harmful inquiries, and increase the overall model security (Shu et al., 2025). Backdoor attribution can regulate attack mechanisms (Yu et al., 2025) and SelfIE can erase harmful responses and replaces unwanted ethical steering (Chen et al., 2024).

Interpretability tools can also be used for debugging as well as explainability and transparency applications (Shu et al., 2025). Millions of features are documented using automated feature labelling pipelines (Paulo et al., 2025), and feature impact on outputs are mapped using attribution graph visualisation tools (Laptev et al., 2025). Research and learning settings will need interpretability pipelines as well (Marks et al., 2025).

Ethical decision making and bias reduction will resolve important social issues (Shu et al., 2025). This will govern toxicity and emotional reactions. We can oversee ethical decision-making. Attribution graphs help us comprehend which internal concepts impact the final outputs.

Although it has improved, interpretable feature extraction has challenges of scalability, accuracy, reliability, generalisation and implementation (Dunefsky et al., 2024; Marks et al., 2025; Shu et al., 2025). SAEs are computationally expensive to train on billion-parameter models and expensive (Gur-Arieh et al., 2025; Shu et al., 2025). The full interpretability is difficult to be achieved, and certain latents cannot be easily interpreted (Paulo et al., 2025). Lastly, there are trade-offs between interpretability and performance, where the costs of computation have an impact on the practical deployment (Gur-Arieh et al., 2025).

2.7 Synthesis

Most studies agree that the best method for controlling model behaviours are sparse autoencoders. On the other hand, they disagree on how to implement, evaluate and deploy these methods the best possible way. SAEs are used on most studies since they are reliable in the

separation of overlapping internal representations into interpretable single purpose features. Nevertheless, researchers use SAEs in different ways (different sparsity mechanisms) and this can have effects interpretability, accuracy, and cost. So there is agreement on the tool, but not on the best setting for it. The other approaches (eg. transcoders, SelfIE) do not replace SAEs, they simply address different issues. SAEs struggle with high density MLP layers, but transcoders can assist there. SelfIE concentrates on explaining representations, instead of obtaining them. But together they widen the scope of analysis.

Input based methods explain what triggered the feature, whereas output based methods what the feature does afterwards. Each capture only half the story, only together do we get the full picture. Big variations in intervention success rates can be attributed to the target. Artificial backdoors can be identified and eliminated with ease since they are localised and there is few of them. Natural false correlations are more difficult since they are spread out on a variety of features. Multi-layer interventions are better since the features change on layers and create circuits, as opposed to nodes. The size of the models is also important: with larger models, controlling is easier since concepts become cleaner. In smaller models, a full SAE analysis is feasible, in medium sized models we can only manage with a partial or selective SAE analysis, and for larger models we need to resort to different methods such SelfIE only. This explains why most do not rely solely on SAEs and must combine other fast and scalable methods with deeper offline analysis on smaller models. It is now being understood in the field that interpretability is not always a measurable property. Automated metrics scale well, but human judgment is needed for verification and more specific cases. There is an evident trend in the field in favour of combining more than one approach, moving towards an integrated pipeline that combines feature extraction and description, cross layer tracking and targeted intervention.

This literature review has established that sparse autoencoders are the primary method for extracting interpretable features from LLMs, but their effectiveness depends strongly on architectural choices (L1 vs. TopK vs. BatchTopK vs. JumpReLU), training configuration, and evaluation methodology. Two gaps in the reviewed literature are particularly relevant to the present work. First, all reviewed studies apply SAEs to models trained on general internet corpora. No published work examines what SAEs learn when applied to a model process-

ing domain-specific text, whether the extracted features encode genuine domain knowledge or merely the surface conventions of domain-specific writing. Second, no reviewed work tracks the corpus provenance of individual features in a multi-source training setting, making it impossible to determine whether a feature's activation signal comes from one type of text (e.g., clinical QA) or another (e.g., research abstracts). This work attempts to addresses both gaps.

3 Methodology

3.1 Pipeline Overview

This capstone project is an end-to-end mechanistic interpretability pipeline made for a domain specialised language model. There are five stages: (i) activations collection from a pre-trained instruction-tuned model on biomedical data; (ii) BatchTopK SAE training on those activations; (iii) feature analysis using three complementary methods; (iv)quality scoring, monosemanticity quantification and a two stage feature labelling; (v) cross-layer comparison of feature representations. Alongside the pipeline is an interactive FastAPI server for behaviour control and feature exploration. The primary analysis target throughout is layer 12. Layers 4 and 8 are trained identically in parallel solely to support the cross-layer analysis in phase (v).

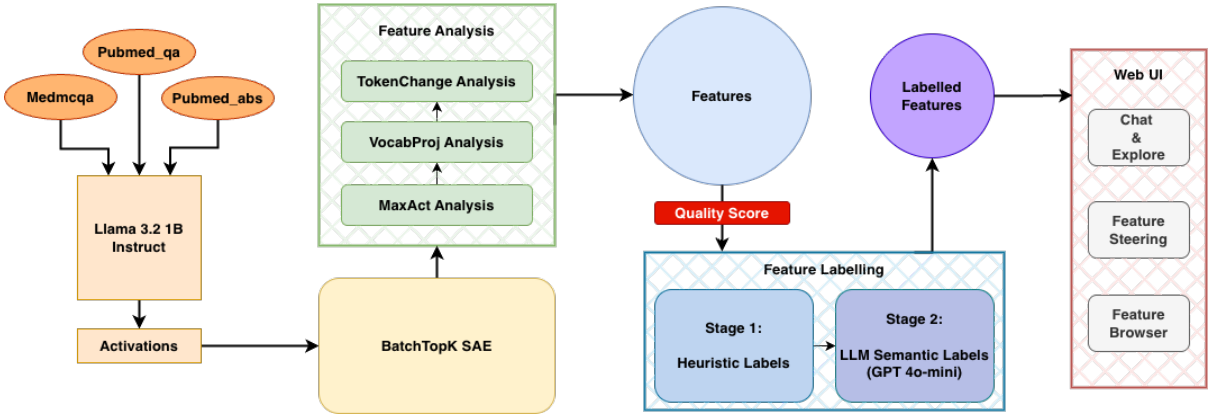


Figure 1: End-to-end single layer pipeline architecture. Biomedical text from three sources, chat-formatted and used to collect residual stream activations from Llama 3.2 1B Instruct. A BatchTopK SAE is trained on these activations; extracted features are then analysed in three steps, and after passing the quality score, they are labelled in two stages. The behaviour control interface is to interact with all results.

3.2 Model Selection

Activations are extracted from `meta-llama/Llama-3.2-1B-Instruct` (hereafter Llama 3.2 1B) (Grattafiori et al., 2024; "Hugging Face Llama 3.2 1B Instruct Documentation", n.d.), loaded and hooked via TransformerLens (Nanda & Bloom, 2022). The instruction-tuned model was chosen over the base model because, the activations we collect from it follow a more

conversational format rather than raw text. If we were to train our SAE with an Instruct-tuned model on plain text activations, the features would be misaligned with the representations the model actually used during generation.

The model has a residual stream dimension of $d_{\text{model}} = 2048$ across 16 transformer layers.

3.3 Biomedical Dataset Construction and Activation Collection

3.3.1 Data Sources

The training corpus is based off three biomedical sources. Each contributes to a third of the total sample target (500,000).

1. **Medical MCQ**: MedMCQA (Pal et al., 2022), a dataset of Indian medical licensing exam questions. Each item is formatted as a question with four answer options and an expert explanation.

2. **PubMed QA**: The `pqa_artificial` split of PubMedQA (Jin et al., 2019), consisting of biomedical research questions with long-form answers derived from PubMed literature.

3. **PubMed Abstracts**: The ccdv/pubmed-summarization dataset (Cohan et al., 2018). Each abstract is split at its midpoint; the first half is presented as a summarisation prompt and the second half as the answer.

All three sources are streamed via Hugging Face Datasets ("Hugging Face Stream Documentation", n.d.), so no full download of corpora are required.

3.3.2 Chat Template Formatting

Each sample is wrapped in the Llama 3.2 Instruct chat template before tokenisation:

```
<|begin_of_text|><|start_header_id|>user<|end_header_id|>

{question}<|eot_id|><|start_header_id|>assistant<|end_header_id|>

{answer}<|eot_id|>
```

This is done to ensure the token distribution during training matches the format the model processes during inference. Every sample also carries a source identifier string (`medmcqa:N`, `pubmed_qa:N`, `pubmed_abs:N`), which we propagate through all the following stages, enabling the per-feature source attribution.

3.3.3 Tokenisation and Activation Extraction

The text samples are tokenised using TransformerLens's `model.to_tokens`. Longer text samples are truncated to `max_tokens` tokens, while empty or whitespace-only samples are ignored. We use `run_with_cache`, caching only the relevant hook `blocks.{layer}.hook_resid_post` with a `names_filter` argument. No extra memory will be allocated for the intermediate computations. The shape of the cached tensor is $[B, T, d_{\text{model}}]$. After filtering out padding tokens with the tokeniser padding token id, all other rows are flattened into a matrix $[N, d_{\text{model}}]$ (N is number of tokens). Then for every sample we assign its source ID so the activation matrix knows which dataset the data came from.

The activations are not stored as one in memory. Instead, the pipeline collects the activations as sequential chunks (`chunk_0.pt`, `chunk_1.pt`, ...) in float16 format once a buffer reaches a size limit. This mechanism makes the entire 500,000 example run feasible with the current amount of RAM we have. The index for each source file is kept as a global tensor, and the sources' names are stored in a string list.

3.4 BatchTopK Sparse Autoencoder

3.4.1 Architecture

Let $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$ denote a single residual stream activation vector. The SAE has the following encoder–decoder structure:

Encoder:

$$\mathbf{z} = W_{\text{enc}}(\mathbf{x} - \mathbf{b}_{\text{pre}}) + \mathbf{b}_{\text{enc}}, \quad \mathbf{h} = \text{TopK}(\mathbf{z}, k),$$

where $W_{\text{enc}} \in \mathbb{R}^{d_{\text{hidden}} \times d_{\text{model}}}$, $\mathbf{b}_{\text{pre}} \in \mathbb{R}^{d_{\text{model}}}$ is a learned pre-encoder bias, $\mathbf{b}_{\text{enc}} \in \mathbb{R}^{d_{\text{hidden}}}$ is the encoder bias, and $\text{TopK}(\mathbf{z}, k)$ retains the k largest positive entries of $\mathbf{z}$, zeroing all others.

Decoder:

$$\hat{\mathbf{x}} = W_{\text{dec}}\,\mathbf{h} + \mathbf{b}_{\text{dec}} + \mathbf{b}_{\text{pre}},$$

where $W_{\text{dec}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{hidden}}}$. Each column $\mathbf{d}_j \in \mathbb{R}^{d_{\text{model}}}$ of W_{dec} is the *feature direction* for feature j in the residual stream. The hidden dimension is $d_{\text{hidden}} = 4 \times d_{\text{model}} = 8{,}192$ and the sparsity aimed for is $k = 64$ active features per token.

3.4.2 BatchTopK Activation

As explained in the literature review, TopK activates exactly k features for each data point. Now BatchTopK on the other hand, activates the top $B \times k$ activations across all B data points, calculating the $(B \times k)$-th activation before setting its value as a common threshold; all activations higher than this are activated while others are zeroed out. There are several advantages of such an approach compared to TopK. individual inputs can use more or fewer than k features depending on their intrinsic complexity (variable per-sample sparsity), and each feature receives gradient signal from more training steps per epoch, reducing the dead feature rate. During inference the model reverts to standard per-sample TopK for deterministic outputs.

3.4.3 Initialisation

Three initialisation decisions are made before training begins:

1. **Decoder initialisation.** W_{dec} is initialised with Kaiming uniform initialisation and immediately column-normalised to unit norm.

2. **Encoder initialisation.** W_{enc} is set to $W_{\text{dec}}^{\top}$, following Gao et al. (2024), so that encoder and decoder directions are initially parallel. This reduces the number of dead features at the start of training compared to random initialisation.

3. **Pre-encoder bias initialisation.** $\mathbf{b}_{\text{pre}}$ is set to the mean of a representative sample of training activations before the first gradient step. This centres the encoder inputs on the empirical activation distribution, a data-driven approximation of the geometric median.

3.4.4 Training Objective

The training loss has two terms:

$$\mathscr{L} = \underbrace{\|\hat{\mathbf{x}} - \mathbf{x}\|_2^2}_{\mathscr{L}_{\text{recon}}} + \alpha \underbrace{\|\hat{\mathbf{x}}_{\text{aux}} - (\mathbf{x} - \hat{\mathbf{x}})\|_2^2}_{\mathscr{L}_{\text{aux}}},$$

where $\alpha = 1/32$. The auxiliary term $\mathscr{L}_{\text{aux}}$ selects up to $k_{\text{aux}} = \min(256, d_{\text{hidden}} - |\text{fired}|)$ features *not* in the main top-k selection (i.e. features that didn't fire on this step) and trains them to reconstruct the current residual $\mathbf{x} - \hat{\mathbf{x}}$. Gradients from $\mathscr{L}_{\text{aux}}$ propagate through these features alone so that the reconstruction path is left untouched. This prevents dead features from forming as we keep giving gradient to the features that fire the least.

3.4.5 Optimiser and Schedule

The SAE is trained for 8 epochs using the Adam optimiser (Kingma & Ba, 2017) with a learning rate of 10^{-4}. The learning rate then decays to 10% of its initial value, using a cosine annealing schedule. Gradients are clipped to a max norm of 1.0 at every step to prevent training instability. Decoder columns are renormalised to unit norm after each optimiser step. Activation chunks are also shuffled randomly at the start of each epoch so that the SAE sees training data in a varied way rather than in fixed order of collection.

3.4.6 Training Diagnostics

Two metrics are tracked and logged per epoch to monitor training health:

Dead feature count. An exponential moving average (EMA) of the per-feature firing indicator $\mathbb{1}[h_j > 0]$ is updated every step: $\text{EMA}_j \leftarrow 0.99 \cdot \text{EMA}_j + 0.01 \cdot \mathbb{1}[h_j > 0]$. Features whose EMA falls below 10^{-5} are classified as dead and counted at the end of each epoch.

Feature usage entropy. At the end of each epoch, the Shannon entropy of the feature usage distribution is computed from the EMA firing rates:

$$H_{\text{usage}} = -\sum_{j:\,\text{EMA}_j > 0} \bar{p}_j \log_2 \bar{p}_j, \quad \bar{p}_j = \frac{\text{EMA}_j}{\sum_{j'} \text{EMA}_{j'}}.$$

A normalised form $H_{\text{norm}} = H_{\text{usage}} / \log_2(d_{\text{hidden}})$ is also recorded, ranging from 0 (one feature dominates) to 1 (perfectly uniform usage). Together these track whether the dictionary is collapsing toward a small set of dominant features across training.

3.5 Feature Analysis Pipeline

After training, the pipeline characterises each feature using three methods executed in the following order.

3.5.1 Pass 1: Basic Feature Statistics

The first pass computes three scalar statistics for every feature j by iterating over all activation chunks:

- **Activation frequency:** $\text{freq}(j) = \Pr(h_j > 0)$, estimated using the amount of token positions a feature j fires on.
- **Mean activation:** $\bar{h}_j = \mathbb{E}[h_j \mid h_j > 0]$, the mean activation conditioned on firing.
- **Maximum activation:** $\max_j = \max_n h_j^{(n)}$ over all token positions n.

These statistics will be used as filters in the quality scoring stage before labelling.

3.5.2 Pass 2: MaxAct Input-Centric Interpretation

MaxAct captures the tokens (and their contexts) from the input that activate a feature the most. A priority queue is maintained for every feature and it is of size M. The following record is appended to the queue when $h_j^{(n)} > h^*$. The record is composed of the following elements: the token string, the token id, the activation score, snippet of surrounding tokens (for context), the global token ID, and the source dataset ID.

After collection, a four-layer pipeline ensures the coherence of our MaxAct examples:

1. **Special-token exclusion.** All special tokens, header tokens, and reported IDs are removed as they are simply just noise.

2. **Surface-form deduplication.** If a token string has already appeared among the retained examples for this feature, we skip it entirely, preventing a single high-frequency subword from dominating the list.

3. **Per-source cap.** A max of two samples are kept per source document (identified by its source string ID). This ensures that an outlier document cannot take up all the top-M positions.

4. **Context-window deduplication.** Only the first 100 characters are considered in comparison with previously saved context strings. This skips duplicate samples from the same section of the document.

The retained source IDs are the foundation of per-feature source attribution.

Additionally, we record for each feature the mean and standard deviation of the token position within the sequence (`position_mean`, `position_std`). This is how we get positional features during heuristic labelling: features that fire disproportionately at fixed sequence positions (low `position_std` relative to sequence length).

3.5.3 Pass 3: VocabProj Output-Centric Interpretation

VocabProj is an estimation of which vocabulary tokens each feature promotes by projecting its decoder direction onto the model's unembedding matrix $W_U \in \mathbb{R}^{d_{\text{model}} \times |\mathcal{V}|}$:

$$\ell_j = \mathbf{d}_j^\top W_U \in \mathbb{R}^{|\mathcal{V}|}.$$

To determine the contribution of each feature over its base value, the mean logit across the vocabulary is subtracted separately for each feature:

$$\tilde{\ell}_j = \ell_j - \bar{\ell}_j, \quad \bar{\ell}_j = \frac{1}{|\mathcal{V}|} \sum_v \ell_{j,v}.$$

The top-k tokens by $\tilde{\ell}_j$ with their corresponding logit values are stored as the feature's vocabulary projection. With this method the retained tokens reflect what the feature specifically

promotes rather than the most frequent subwords in the vocabulary. To keep memory bounded we perform computation in chunks of 512 features.

3.5.4 Pass 4: TokenChange Causal Interpretation

TokenChange captures the causal relationship between a feature and the model output by inserting the decoder direction of the feature into the residual stream during a forward pass. For each of up to 5 MaxAct contexts c associated with feature j, a TransformerLens hook at `blocks.{layer}.hook_resid_post` adds the following perturbation at the token position where the feature originally fired:

$$\mathbf{a} \leftarrow \mathbf{a} + \lambda_j \cdot \mathbf{d}_j, \quad \lambda_j = \max\left(\gamma \cdot \bar{h}_j,\ 1.0\right),$$

where $\gamma = 5.0$ is a fixed scale multiplier and $\bar{h}_j$ is the feature's mean activation over the training dataset. The perturbation magnitude is therefore proportional to the feature's typical activation strength, ensuring that weakly-firing features still receive a minimum injection of one unit of their decoder direction. The next-token probability distribution is computed for the steered and unsteered passes, and the shift is measured via KL divergence:

$$\mathrm{KL}_j(c) = D_{\mathrm{KL}}\left(p_{\text{steered}}(\cdot \mid c) \,\middle\|\, p_{\text{baseline}}(\cdot \mid c)\right).$$

This is averaged across all contexts used to produce a mean $\overline{\mathrm{KL}}_j$ and standard deviation σ_{KL_j}. The top promoted and suppressed tokens under the intervention are recorded. TokenChange is the method which produces causally consistent outputs at layer 12 since the perturbation passes through all downstream transformer layers until the distribution of output tokens.

3.6 Post-Training Evaluation Metrics

After training and feature analysis, we compute two summary statistics characterise the overall SAE quality.

Explained variance. A single-pass accumulator in float64 precision computes the fraction of activation variance reconstructed by the SAE:

$$\mathrm{EV} = 1 - \frac{\mathbb{E}_n\left[\|\mathbf{x}_n - \hat{\mathbf{x}}_n\|_2^2\right]}{\mathrm{Var}(\mathbf{x})},$$

where $\mathrm{Var}(\mathbf{x}) = \sum_d \mathrm{Var}(x_d)$ is the total variance summed across all d_{model} dimensions, estimated as $\sum_d (\mathbb{E}[x_d^2] - \mathbb{E}[x_d]^2)$ from accumulated sums. Float64 accumulators are used throughout to avoid catastrophic cancellation when subtracting two large squared-sum quantities. Explained variance is the primary reconstruction quality metric and is reported alongside the dead feature count in the final summary as well as other training configuration metadata.

Dead feature rate. The fraction of the total 8,192 dictionary features whose EMA firing rate remained below 10^{-5} at the end of training (as a percentage). This is the primary capacity utilisation metric.

3.7 Monosemanticity Quantification and Feature Quality Scoring

3.7.1 Shannon Entropy as Monosemanticity Proxy

For each feature j, let $p_j(t)$ be the normalised frequency of token t among the feature's MaxAct examples. Basically, how similar are these examples to each other. The monosemanticity of feature j is quantified by the Shannon entropy of this token distribution:

$$H(j) = -\sum_t p_j(t) \log_2 p_j(t) \quad \in [0, \log_2 M],$$

where M is the number of MaxAct examples collected. A low $H(j)$ means the feature fires on similar tokens, a tight, semantically coherent cluster of them (monosemantic); a high $H(j)$ indicates the opposite. The feature is firing on many unrelated tokens (polysemantic), which is not inherently bad, a "drug" polysemantic feature firing on different names of drugs is still important. This metric is agnostic to semantic content. It's computed for every feature from the MaxAct pass and requires no human annotation.

3.7.2 Multi-Component Quality Score

Not all features are labelled; they must be ranked using a composite quality score $Q(j) \in [0,1]$ that combines six components: activation frequency (double-weighted), MaxAct coverage, Vo-

cabProj coverage, mean activation strength, token diversity, and normalised entropy:

$$Q(j) = \frac{1}{7}\left(2 \cdot S_{\text{freq}}(j) + S_{\text{cov}}(j) + S_{\text{vocab}}(j) + S_{\text{peak}}(j) + S_{\text{div}}(j) + S_{\text{ent}}(j)\right).$$

Features with activation frequency outside $[10^{-4}, 0.30]$ are excluded from quality ranking and LLM labelling entirely. The full definition of each component is given in Appendix B.

3.8 Feature Labelling Pipeline

3.8.1 Stage 1: Heuristic Classification

The heuristic pass is applied to all features before quality filtering and requires no API calls. Using the MaxAct token set, it classifies features into structural categories:

- **Special-token features:** Features whose MaxAct tokens are predominantly Llama chat template tokens (<|begin_of_text|>, <|eot_id|>, header tokens) are labelled `template_token`.
- **Punctuation features:** Features whose MaxAct tokens consist entirely of punctuation characters are labelled `punctuation`.
- **Digit features:** Features whose MaxAct tokens are numeric strings are labelled `digit`.
- **Positional features:** Features with a low ratio of position standard deviation to sequence length. Meaning they fire predominantly at a fixed position in the sequence so we label them `positional`.

These structural features are assigned the previous labels and a (`high`) confidence without any further processing.

3.8.2 Stage 2: LLM Semantic Labelling

Features passing the quality filter are submitted in batches to GPT-4o-mini (OpenAI, 2024) for semantic labelling. The prompt for each feature contains three elements:

1. **Multi-quantile MaxAct examples.** Tokens are sampled from 3 activation quantile tiers: high, mid, and low. Each token is shown with its activation strength and a surrounding

context snippet. If we were to sample the full distribution of activations, rather than just the top-K, the model would have access to edge cases that may contradict top signal and mislead it.

2. **Mean-centred VocabProj tokens.** The top vocabulary tokens from the mean-centred projection $\tilde{\ell}_j$ are provided with their raw logit values, to give an output-centric view of what the feature promotes.

3. **Source attribution fractions and metadata.** Finally, the rest of the feature metadata, such as the source breakdown, activation frequency and mean activation. As additional information, so the labelling model can account for corpus-specific vocabulary patterns when interpreting ambiguous tokens.

The labelling model returns a short natural language description and a confidence level: `high`, `medium` or `low`. GPT-4o-mini has priority when labelling over heuristic labels when both are produced for the same feature.

3.8.3 Semantic Categories

All labelled features are assigned to one of eleven categories using keywords classifying: Structural/Linguistic, Research Methodology, Clinical/Diagnostic, Pharmacology, Biochemical/Molecular, Anatomical, Epidemiological, Oncology, Microbiology/Infectious, Mental Health/Psych as well General/Other for the more niche features. Every category, description and matching keywords are given in Appendix C.

3.9 Per-Feature Source Attribution

The source ID propagated from the first stage (data loading) through tokenisation and activation collection means that every MaxAct example for a feature j has a known corpus origin. For each feature, the pipeline computes the fraction of MaxAct examples from each source:

$$\text{SrcBreakdown}(j) = \left\{ s \;:\; \frac{\#\{\text{MaxAct}(j) \text{ from source } s\}}{\#\{\text{MaxAct}(j)\}} \right\}_{s \in \{\text{medmcqa}, \text{pubmed_qa}, \text{pubmed_abs}\}}.$$

This breakdown is stored as a first-class field on every `FeatureInfo` object and is included in the LLM labelling prompt. It also constitutes an independent empirical line of evidence for the main finding: a feature labelled as a Research Methodology feature based on its token distributions can be cross-validated by verifying that its MaxAct examples originate predominantly from PubMed Abstracts, without any reliance on the LLM labelling step.

3.10 Cross-Layer Analysis

SAEs are trained independently at layers 4, 8, and 12 using identical architecture and training configurations. All three SAEs are then compared using pairwise cosine similarity between decoder feature directions:

$$\text{sim}(j,\, j') = \frac{\mathbf{d}_j^\top \mathbf{d}_{j'}}{\|\mathbf{d}_j\|_2 \, \|\mathbf{d}_{j'}\|_2}, \quad j \in L_A,\ j' \in L_B.$$

For each feature j in layer A, the maximum similarity to any feature in layer B is recorded. The mean of these maximum similarities (`mean_max_similarity`) characterises overall structural overlap between the two layers. Features with a maximum similarity above 0.7 to any feature in another layer are classified as structurally shared across layers. A many-to-one convergence pattern: when multiple layer-A features share the same best-matching layer-B feature. It can be identified by grouping layer-B features by their most frequent layer-A match and recording the size and similarity distribution of each group. This provides direct evidence of hierarchical feature composition: fragmented representations at shallower layers converging onto integrated features at deeper layers.

Throughout the cross layer analysis, each layer goes through its respective feature analysis pipeline (MaxAct, VocabProj, TokenChange, quality scoring, and labelling). We can then directly compare the different feature categories, dead feature rates, Shannon entropy distributions, and source attribution profiles across layers.

3.11 Behaviour Control Interface

The behaviour control interface uses FastAPI server with REST API and a static web frontend to contact the SAE and labelled features. It is organised into four tabs: Chat, Steer, Explore and Features.

3.11.1 Chat and Explore

The **Chat** tab accepts a user message and runs a full attributed forward pass. The prompt is tokenised, run through Llama 3.2 1B, and the residual stream activations at the target layer are encoded by the SAE. The top activated features are ranked by activation magnitude, with labelled features prioritised over unlabelled ones. Similarly to any conversational language model, it generates a response token-by-token; the generated activations are similarly encoded and the top output features are extracted. Both input and output feature attributions are returned alongside the streamed response tokens via Server-Sent Events (SSE).

The **Explore** tab provides a token-level circuit view of an arbitrary input. The top features activating on each token are returned as a bipartite token-to-feature graph, rendered as a D3.js Sankey diagram. A feature table below the diagram lists activated features with labels, activation values, confidence badges, and highlighted token context.

3.11.2 Feature Steering

The **Steer** tab allows any labelled feature to be amplified or suppressed during generation. The user selects one or more features and sets a strength $s_j \in [-50, +50]$ for each. The individual feature strengths are combined into a single steering direction in model space:

$$\mathbf{v} = W_{\text{dec}}\,\mathbf{s}, \quad \hat{\mathbf{v}} = \frac{\mathbf{v}}{\|\mathbf{v}\|_2 + \varepsilon},$$

where $\mathbf{s} \in \mathbb{R}^{d_{\text{hidden}}}$ is the sparse vector of user-specified strengths and W_{dec} is the decoder weight matrix. The resulting unit direction $\hat{\mathbf{v}}$ is scaled relative to the residual stream norm observed on the input prompt:

$$\delta = \hat{\mathbf{v}} \cdot \|\mathbf{a}\|_2 \cdot \min(s_{\max} \cdot 0.01,\ 0.25)\,,$$

where $s_{\max} = \max_j |s_j|$ and $\|\mathbf{a}\|_2$ is the mean residual stream norm over input positions. The 0.25 ceiling prevents the intervention from exceeding 25% of the activation norm, avoiding degenerate outputs. This perturbation is added at every token position via a TransformerLens hook during autoregressive generation. The output is a normal generation, without intervention, alongside the steered generation.

3.11.3 Feature Browser

The **Features** tab gives the user full access to the complete labelled feature browser (48 per page). There are confidence filters (High / Medium / Low) on top as well as a search bar for feature ID or label. As a feature is selected, we get its full analysis: the MaxAct context with activations, VocabProj with logit, TokenChange promoted and suppressed tokens with mean KL divergence, the source attribution breakdown, and quality score.

4 Experiments

This chapter provides information regarding the pipeline of the experiments for training sparse autoencoders (SAEs). SAEs are trained to extract and annotate semantic features from the residual stream and conduct cross-layer feature analysis. Initial experiments were conducted using an MPS Apple machine (Macbook Pro M4). The final multi-layer training was conducted using a university high-performance computing (HPC) CPU cluster. The entire codebase can be found at ("Github Capstone Project", n.d.).

4.1 Experimental Design Overview

The process of experiments follows the same pipeline proposed in the methodology. Every step of the pipeline is repeated for more than fifty trials. The reasons behind each configuration decision include both theoretical arguments from the mechanistic interpretability literature and practical insights from all previous experiments.

It was decided that a 1B parameter model was a good compromise between interpretability and the capacity to extract information, given that small models such as GPT-2 (Radford et al., 2019) (124M) produced relatively coarse features during preliminary tests, whereas larger models above 7B parameters were not feasible due to computational limits.

4.2 Dataset Configuration

The combined dataset has approximately 453,000 samples (90.7% of the 500,000 target), drawn from three sources: MedMCQA (166,666 samples, instruction-formatted multiple-choice QA), PubMed QA (166,666 samples, context passages only with `Question:/Answer:` prefixes stripped), and PubMed Abstracts (119,924 samples, raw prose). After the tokenisation and removal of padding, this yielded 46.3M tokens across 46 on-disk chunks, 4.3$\times$ more than the previous run. A three-level fallback chain was implemented for MedMCQA loading to handle dataset availability failures silently.

Dataset Breakdown by Source

Source	Samples	Tokens (approx.)	Avg seq len (tokens)
MedMCQA	166,666	~17,036,479	~102
PubMed QA	166,666	~17,036,479	~102
PubMed Abstracts	119,924	~12,258,545	~102

Figure 2: The biomedical training datasets, with sample count, token count, and mean sequence length after chat-template formatting and tokenisation.

4.3 SAE Architecture and Training

The first tests that were conducted on L1 regularisation on the GPT-2 medium (355M) model during the early phases of this study showed a problematic trend, as the L1 increased to ensure sparsity within the network of 2-3%, the amount of dead neurons kept growing, when decreased we would get an average L0 sparsity value highly above goal. After trying over a dozen different values for L1 ranging from 5×10^{-3} to 50, no combination was found that would satisfy the criteria of low L0 sparsity and low dead neurons. This led us to choose the TopK activation method proposed by Gao et al. (2024). Neuron resampling (Bricken et al., 2023) was tried initially, but proved to be ineffective therefore not considered in the final design.

Table 2 reports the final per-layer training metrics from the cross-layer run.

Table 2: Per-layer SAE training summary. All three SAEs were trained on 46.3M tokens (8 epochs, K=64, d_{hidden}=8,192).

Metric	Layer 4	Layer 8	Layer 12
Dead features	1,372 (16.7%)	3,861 (47.1%)	1,264 (15.4%)
Alive features	6,820	4,331	6,928
Explained variance	41.4%	41.6%	47.2%
Final loss	0.00132	0.00253	0.00765
Entropy (normalised)	0.840	0.831	0.865

Layer 12 shows the maximum explained variance (47.2%) and minimum percentage of dead features (15.4%), showing that there are more structured and less isotropic activations within the deep residual stream that the SAE can decompose successfully. Layer 8 is an anomaly, in that the number of dead features suddenly increases to 4,428 during epoch 3 and remains high with the minimum being 3,978, leading to a dead rate of 47.1%, which is about three times

higher than those seen in layers 4 and 12. The mid-layer residual stream appears to be located in a highly entangled activation space, making it difficult for the SAE to discover sparsity. The loss function in each layer decreases similarly, exhibiting a fast drop (epoch 1) followed by slow convergence; at epoch 8, the loss in Layer 12 was still decreasing slightly.

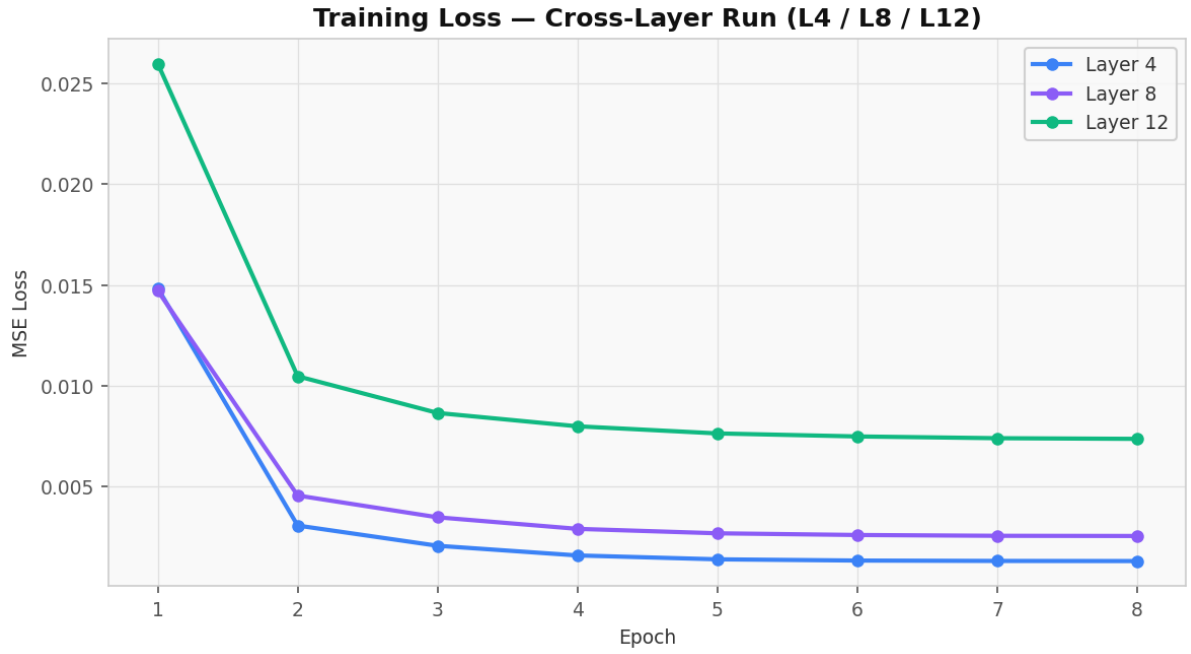


Figure 3: Reconstruction loss across 8 training epochs for the final cross-layer run (Attempt 53, Llama 3.2 1B Instruct, Layers 4, 8, and 12, K=64, d_{hidden}=8,192, 46.3M tokens).

4.4 Feature Labelling Pipeline

In the first dozen attempts, features were sent individually to GPT 4o, one API call per feature. This was too expensive to scale, so batching was implemented. The top 1000 features by activation frequency were sent in batches of 50 to cut costs. Results were poor, with only 46 labelled features, 4 being high confidence. The problem was the selection criterion. The most frequently firing features are polysemantic. This was confirmed in a next attempt when, for example, “Feature 3521” activated extremely high on every single token in circuit analysis.

A quality heuristic was then implemented. Frequency-based selection was replaced with a composite quality score combining frequency, MaxAct coverage, VocabProj coverage, activation strength, and token diversity. Features firing too rarely (<0.5%) or too often (>15%) were excluded entirely, effectively removing features like “3521”. Results were promising, 186

labelled features, with 116 high confidence, a times 4 increase.

Mean-centring for VocabProj: Raw VocabProj tokens were dominated by frequent subwords regardless of the feature type. Removing the average logit of each feature so each token reflected promoted above baseline changed this: nationality and country features now cleanly clustered from one another. Label accuracy increased to 19.5% with 176 confident labels.

Finally, automatic labelling based on heuristics and entropy-based scores were applied, achieving even more impressive results. A heuristic-based pass was introduced before the LLM labelling stage to detect the positional, punctuation, and digit features automatically. The quality score was further improved by adding a new component: Shannon entropy across the MaxAct tokens. Monosemantic features were now encouraged. The new setup achieved 1,355 labelled features (16.5% label rate), 77.8% of which were considered high-confidence features.

After the switch to Llama 3.2 1B Instruct on medical data, however, the pipeline crashed. This time, it outputed 54 labels at 5.7%. Running it a dozen more times to pinpoint the cause revealed that MedMCQA format tokens were polluting MaxAct examples, the frequency threshold was too strict for infrequent medical language, and text activations did not match up with instruction-tuned chat-format representations. To solve these issues, the data had to be wrapped in the Llama chat template, special tokens removed from MaxAct, the frequency threshold lowered from 0.1% to 0.01%, activations sampled from the top, mid, and bottom quantiles instead of only the top quantile, and source breakdown percentages included in the labelling prompt. This brought the pipeline back to life (500k samples, layer 8) with 535 labelled features at 21.0%.

The final pipeline was applied independently to layers 4, 8, and 12 (8 epochs, 46.3M tokens). Layer 4 produced 1,982 labelled features (24.2%), layer 8 produced 1,970 (24.0%), and layer 12 produced 3,373 (41.2%), nearly doubling the other two layers. At layer 12, 62.2% of labels were high-confidence. Every medical category (Clinical, Pharmacology, Biochemical, Anatomical, Epidemiological) grew in proportion from L4 to L12, while General/Other and Structural/Linguistic shrank.

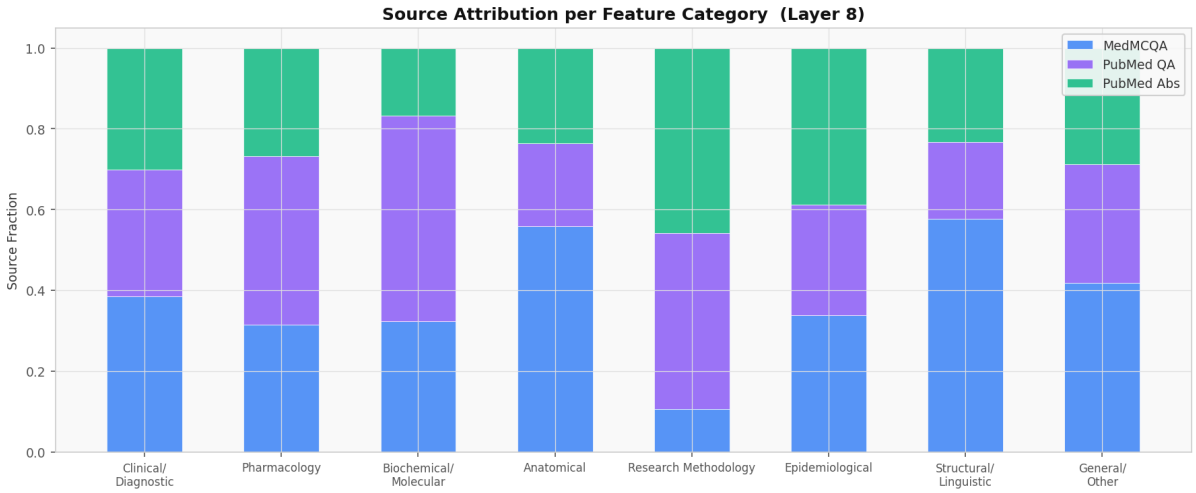


Figure 4: Source attribution fractions per semantic label category, showing the proportion of MaxAct examples drawn from MedMCQA, PubMed QA, and PubMed Abstracts for each cluster. Research Methodology features draw predominantly from PubMed sources; Clinical/Diagnostic features draw predominantly from MedMCQA.

4.5 Cross-Layer Analysis

In the final run, the number of labelled features generated at Layer 12 was 3,390 features (41.4% of the dictionary), out of which 2,071 were labelled with high confidence, 1,073 with medium, and 144 with low confidence. The heuristic labelling provided 514 features (234 positional, 280 token-pattern). The semantic categories of features at Layer 12 are dominated by Clinical/Diagnostic (18.6%), Structural/Linguistic (17.1%), Biochemical/Molecular (6.3%), Pharmacology (5.8%), Anatomical (5.0%), and Research Methodology (5.6%).

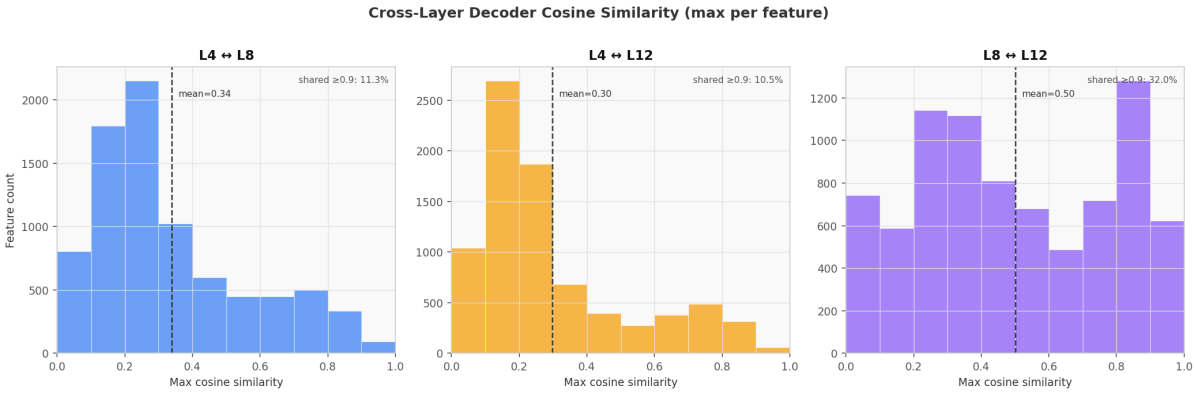


Figure 5: Distributions of max cosine similarity between SAE decoder directions for all layer pairs. Each of the histogram shows us the highest cosine similarity, for every feature in the source to the target layer.

5 Results

5.1 SAE Training: L1 Regularisation vs. TopK Activation

To choose the best possible sparsity method, regardless of model or dataset, we conducted a comparison of L1 and TopK on the same model GPT-2 medium (Radford et al., 2019) trained on OpenWebText (Gokaslan & Cohen, 2019). This comparison spans over the first two dozen runs. With L1 regularisation, the coefficient increased from 5×10^{-3} to 50 but we still failed to achieve target sparsity. A high coefficient gave us a decent L0 around 121 active features per token (8,192 total) but a drastic amount of dead features. Now the low L1 had few dead neurons but a very high sparsity corresponding to 21% feature utilisation (1762 active features). The recommended range in the literature is 1–3% (Bricken et al., 2023).

Switching to TopK activation on GPT-2 medium without changing our hyperparameters resolved this. With $K = 64$, exactly 64 features activate per forward pass, yielding a fixed L0 of 0.78%. Regardless of dead features at least we had a great sparsity. The labelling results on GPT-2 medium with TopK (500k OpenWebText samples, ≈64M tokens) were substantially better than anything achieved with L1: approximately 1,355 features were labelled in total (16.5% of all features). Feature steering experiments on this GPT-2 TopK SAE produced clear and reliable causal interventions. When amplifying sentiments or political concepts, it produced actual shifts in the generated text. These results confirmed TopK as the correct architectural choice for the pipeline.

However, the GPT-2 TopK features, while well-formed and steerable, were broad topical features (geographic names, temporal references, sports outcomes, politics) and this was not a conversational model. This was expected: GPT-2 is a general-purpose, non-conversational model trained on diverse web text, and its internal representations will reflect as such. We wanted domain specificity and a model with richer internal representations.

This drove the switch to Llama 3.2 1B Instruct trained on a biomedical corpus. It was chosen to test the pipeline with a model and training data aligned to a specialised domain. The transition introduced significant new challenges: the larger model (≈3× more parameters than

GPT-2 medium) required substantially more compute per forward pass, making activation collection far more time-intensive (the final 500k-sample Llama run took 81 hours of collection time alone for a single layer). The switch to multi-source medical datasets also introduced formatting artefacts and distribution shifts, and the instruction-chat template formatting of Llama 3.2 Instruct required careful handling to avoid contaminating MaxAct examples with structural tokens. The first dozen Llama runs produced substantially worse labelling results than the final GPT-2 runs, with as few as 38 labelled features at a 5.3% label rate. That was before dataset, architecture, and labelling pipeline changes brought to the system as described below.

Table 3: Comparison of L1 and TopK SAE training runs across model and dataset scales. The final column reports the latest results from the cross-layer run.

Metric	**L1 Best – GPT-2 (Att. 9)**	**TopK GPT-2 (Att. 30)**	**TopK Llama Final (Att. 53)**
Model	GPT-2 medium (355M)	GPT-2 medium (355M)	Llama 3.2 1B Instruct
Dataset	OpenWebText 50k (≈6.4M tok)	OpenWebText 500k (≈64M tok)	Medical 500k (46.3M tok)
Avg L0 sparsity	457 / 6,144 (7.4%)	100 / 8,192 (1.2%)	64 / 8,192 (0.78%)
Dead features	2 (0.03%)	≈3,058 (37%)	1,264 (15.4%) [L12]
Total labelled	≈46	≈1,355	3,390 [L12]
High-conf. labels	4	≈1,054 (77.8%)	2,071 (61.1%)
Label rate (cand.)	<1%	16.5%	41.4%

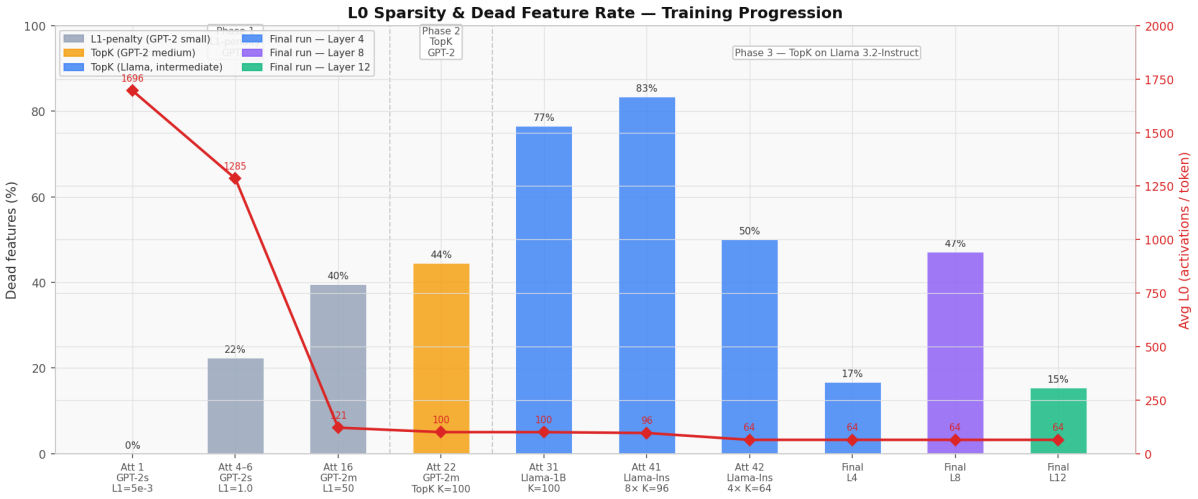


Figure 6: L0 sparsity and dead feature rate across experimental runs, showing the progression from L1-regularised SAEs on GPT-2 through TopK SAEs on GPT-2 to the final Llama 3.2 1B configuration.

5.2 Feature Labelling Results

In the last labelling run (500k samples, Llama 3.2 1B Instruct, Layer 12, 8 epochs), we got 3,390 labelled features among 8,192 overall (41.4

Table 4: Distribution of labelled features by semantic category (Layer 12, 3,390 total).

Category	Features	% of Labelled	Representative example
General/Other	1,227	36.2%	Biological processes and mechanisms
Clinical/Diagnostic	631	18.6%	Clinical syndrome descriptions
Structural/Linguistic	579	17.1%	Positional features, discourse markers
Biochemical/Molecular	213	6.3%	Cellular structure, molecular biology
Pharmacology/Treatment	196	5.8%	Therapeutic drug levels
Research Methodology	191	5.6%	Experimental design terms
Anatomical	169	5.0%	Anatomical origins of conditions
Epidemiological	61	1.8%	Patient demographics
Microbiology/Infectious	52	1.5%	Viral infections, disease transmission
Oncology	38	1.1%	Tumour grading classification
Mental Health/Psych	33	1.0%	Psychiatric disorder terminology

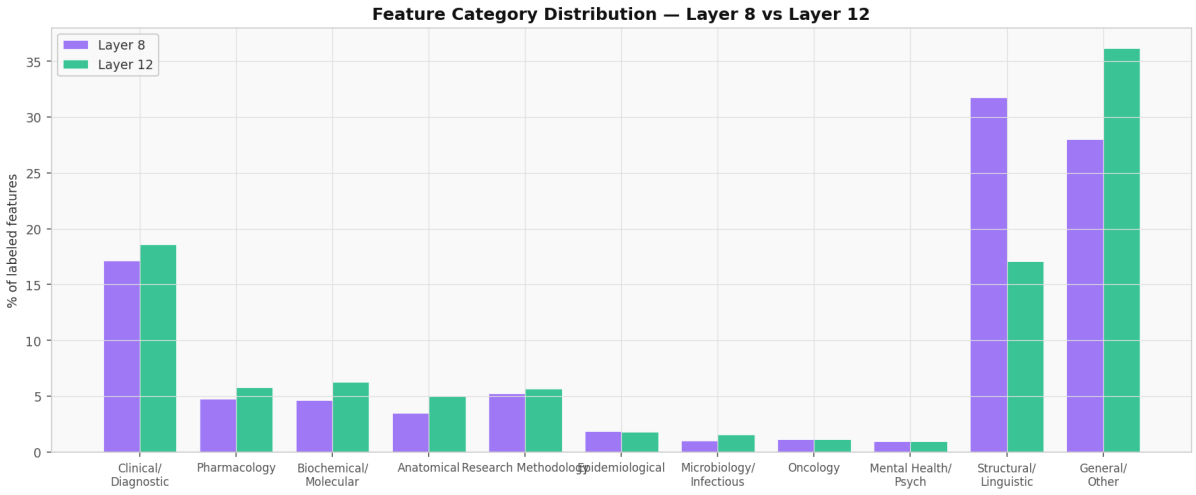


Figure 7: Distribution of labelled features by semantic category at Layer 8 and Layer 12, illustrating the depth-dependent shift from Research Methodology features at mid-layer to Clinical/Diagnostic features at the deeper layer.

5.3 Main Empirical Finding: SAEs Learn the Language of Medical Research, Not Medical Knowledge

The central empirical finding of this capstone project is what SAEs actually learned when trained on a domain-specific instruction-tuned model. At layer 8, where the vast majority of

our early work was conducted, research language was the largest cluster of labelled features. But following a cross layer analysis, it was found that at layer 12, this balance reverses, clinical/diagnostic features overtake methodology, all domain knowledge categories increase while structural linguistic diminish. The depth-dependent shift from writing-convention features to domain-knowledge features is the central finding of this thesis.

This finding is also directly and quantitatively supported by the per-feature source attribution metadata (`source_breakdown`). For each labelled feature, we record the fraction of MaxAct examples originating from each dataset source (MedMCQA, PubMed QA, and PubMed Abstracts). The source breakdown data reveal a clear and consistent pattern: features labelled as Research Methodology draw their MaxAct examples predominantly from PubMed Abstracts and PubMed QA, which are sources dominated by the conventions of scientific writing, while Clinical/Diagnostic features draw more heavily from MedMCQA, which contains clinical reasoning formatted as question-answer pairs. Table 5 illustrates this pattern with concrete examples.

Table 5: Source attribution for representative features, showing corpus provenance of MaxAct activations.

Feature	Label	Category	Source breakdown	VocabProj top
1813	Survey methodology	Research Meth.	pubmed_abs=100%	survey, question
4556	Study participants	Research Meth.	pubmed_abs=100%	participant, incl
5124	Hormonal conditions	Clinical/Diag.	medmcqa=100%	hormone, adren
7063	Medical classifications	Clinical/Diag.	medmcqa=100%	form, type, type
3802	Therapeutic drug levels	Pharmacology	medmcqa=60%, pubmed_qa=40%	levels, concentr
5225	Calcium metabolism	Biochem./Mol.	medmcqa=50%, pubmed_qa=50%	calcium, bone,
7951	Spinal conditions	Anatomical	medmcqa=75%, pubmed_abs=25%	spinal, spine, di

Importantly, this source analysis is performed on the level of provenance of the MaxAct tokens. It is pre semantic label, meaning it is an independent line of evidence for the main finding. It cannot be attributed to labelling bias. Also, this tendency cannot be explained merely by the structure of the dataset, as PubMed sources are present in significant numbers in the dataset, but the Clinical/Diagnostic and Anatomical features show that the model can learn domain-specific knowledge.

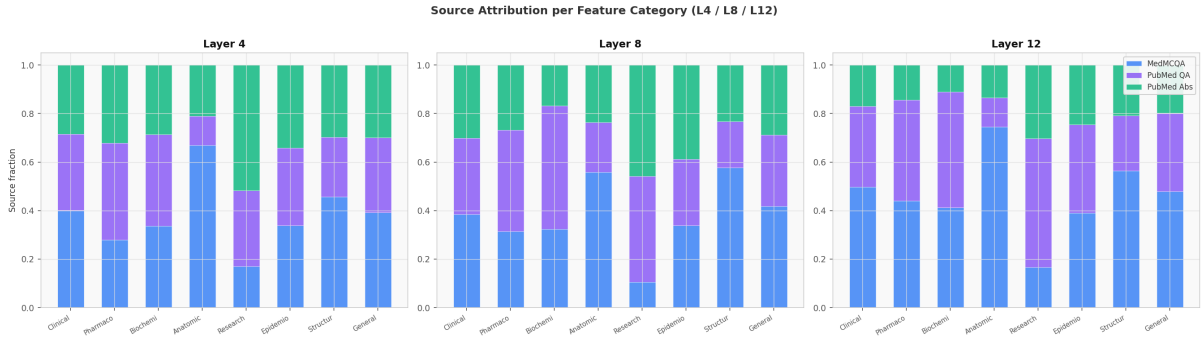


Figure 8: Source attribution breakdown per semantic feature category across all three layers, showing the fraction of MaxAct examples originating from MedMCQA, PubMed QA, and PubMed Abstracts. The pattern is independent of semantic labelling and constitutes a second line of evidence for the main finding.

5.4 Cross-Layer Analysis: Feature Composition and Depth-Dependent Semantic Emergence

SAEs were trained independently on Layers 4, 8, and 12 of Llama 3.2 1B Instruct using the same configuration (8 epochs, K=64, d_{hidden}=8,192, 46.3M tokens). The per-layer training statistics reveal a striking progression with network depth.

Table 6: Per-layer SAE training and labelling statistics (Final run, 8 epochs).

Metric	Layer 4	Layer 8	Layer 12
Dead features	1,372 (16.7%)	3,861 (47.1%)	1,264 (15.4%)
Alive features	6,820	4,331	6,928
Explained variance	41.4%	41.6%	47.2%
Final loss	0.00132	0.00253	0.00765
Labelled features	1,982 (24.2%)	1,970 (24.0%)	3,390 (41.4%)
High confidence	62.6%	55.3%	61.1%
Mean entropy	1.15 bits	1.46 bits	1.65 bits

Layer 12 achieves the highest explained variance (47.2%) and the lowest dead feature rate (15.4%), indicating that the deeper residual stream contains more structured, non-isotropic activations that the SAE can decompose effectively. Layer 8 is a notable anomaly: dead features spike to 4,428 at epoch 3 and never recover below 3,978, resulting in a 47.1% dead rate — roughly three times that of layers 4 and 12. This suggests the mid-layer residual stream sits in an activation space that is highly entangled, making it harder for the SAE to find independent sparse directions.

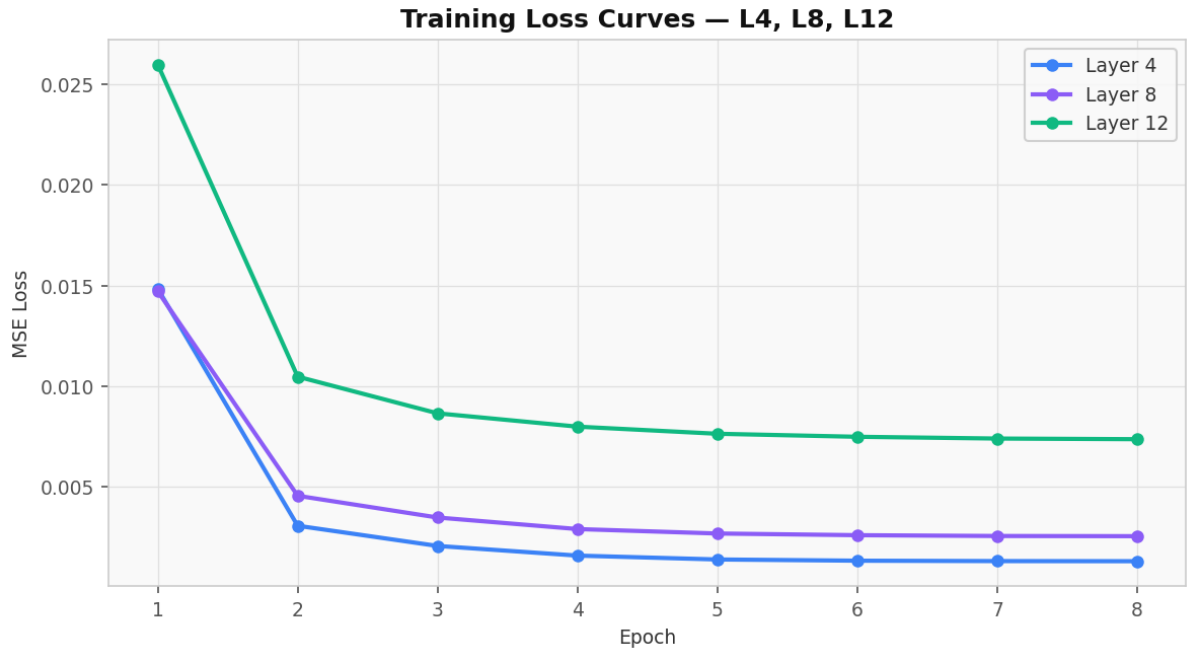


Figure 9: Reconstruction loss per epoch for SAEs trained at Layers 4, 8, and 12. Layer 4 converges fastest; Layer 8 plateaus early with the highest final loss; Layer 12 shows the steepest and most sustained improvement across all 8 epochs.

Qualitatively, each of the three layers represent a unique level of representation:

- **Layer 4**: positional and structural features (22.4% heuristic-positional). The few semantic features are poorly localised. For example: the best kidney/renal feature fires 50% of the time on unrelated organs (hepatic stellate cells, lung injury). Mean entropy is 1.15 bits, reflecting narrow, token-specific firing patterns.

- **Layer 8**: here features are transitioning toward semantic specificity. We are finding decent medical and research methodology features, but they are mostly capturing the structure of medical writing rather than its content. Mean entropy is 1.46 bits.

- **Layer 12**: finally genuine semantic features emerge. Some big feature clusters include Clinical/Diagnostic (18.6%), Pharmacology (5.8%) and Anatomical (5.0%). Mean entropy is 1.65 bits, with 67.3% of labelled features below 2.5 bits, indicating coherent token clusters.

Cross-layer cosine similarity analysis reveals a directional pattern of feature evolution across depth.

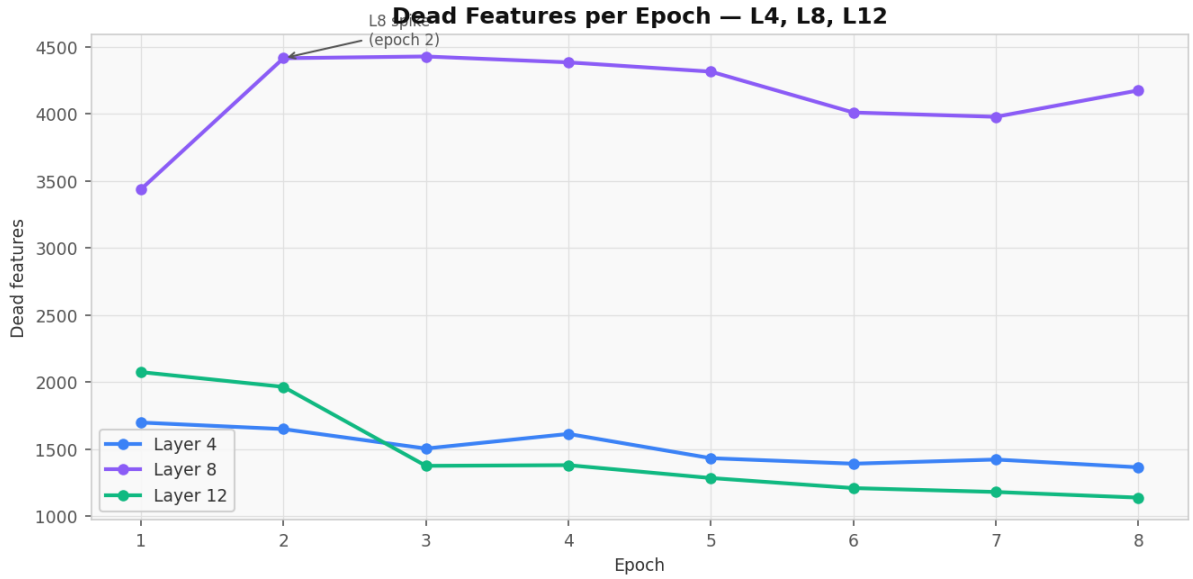


Figure 10: Dead feature count per epoch for Layers 4, 8, and 12. The Layer 8 anomaly — a spike to 4,428 dead features at epoch 3 followed by only partial recovery — is visible in the middle curve and is absent from the other two layers.

Table 7: Cross-layer cosine similarity statistics between SAE decoder directions.

Direction	**Mean max-sim.**	**Shared (>0.7)**	**% of source layer**
L4 → L8	0.339	927	11.3%
L8 → L4	0.502	2,602	31.8%
L4 → L12	0.298	858	10.5%
L12 → L4	0.285	932	11.4%
L8 → L12	0.500	2,624	32.0%
L12 → L8	0.323	1,021	12.5%

Figure 5 shows the full cosine similarity distributions for all three layer pairs.

The directional asymmetry is the most informative structural finding. 11.3% of Layer 4 features survive into Layer 8, but 31.8% of Layer 8 features have a high-similarity precursor in Layer 4, meaning the L4→L8 transition is primarily expansive. On the contrary, 32.0% of Layer 8 features survive into Layer 12, but only 12.5% of Layer 12 features are derived from Layer 8,this transition is not expansive. Layer 12 generates the majority of its features over again. Overall, 88–90% of features at each layer have no near-duplicate in any other layer. Clearly confirming that layers are largely independent.

Many-to-one convergence patterns are visible in decoder weight space: the top convergence target in the L8→L12 comparison (feature L12:3829) is geometrically close to 1,278 Layer 8 decoder directions. However, these top attractor features are dead or near-dead (activation

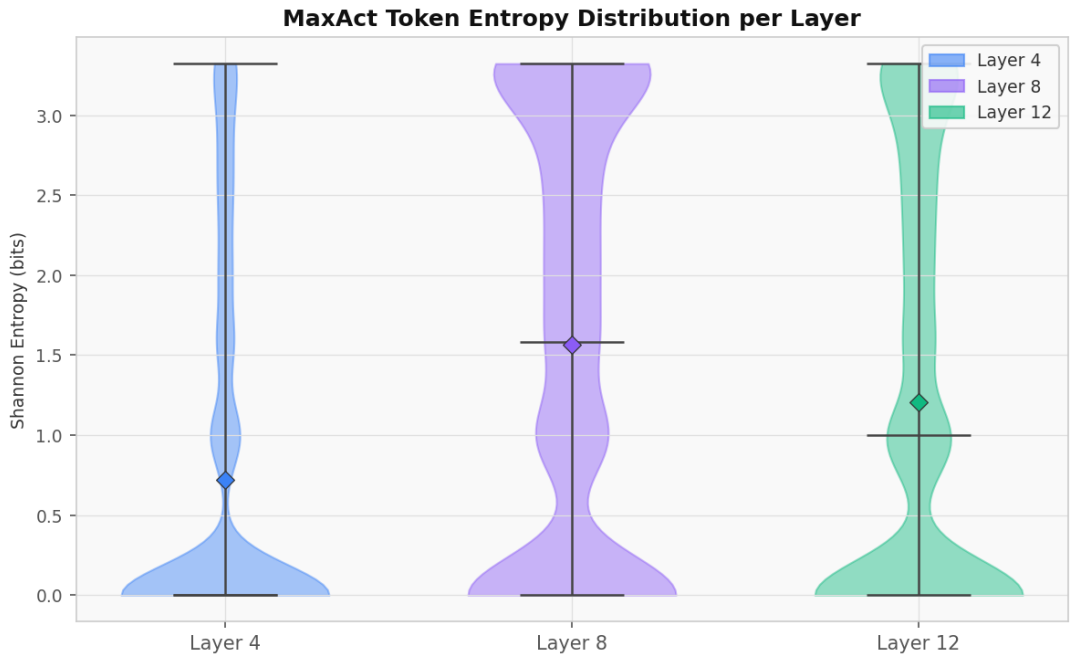


Figure 11: Distribution of Shannon entropy of MaxAct token distributions across labelled features at Layers 4, 8, and 12. Lower entropy indicates tighter, more monosemantic token clusters. The rightward shift with depth reflects increasing semantic specificity.

frequency $< 10^{-5}$), meaning the convergence reflects geometric clustering of dead decoder columns rather than active semantic integration. The convergence is a real weight-space phenomenon. Dead features cluster in specific directions rather than being uniformly distributed.

These cross-layer findings support and refine the main empirical finding. The shift from discourse-level methodology features at Layer 8 to genuine medical entity features at Layer 12 is not a property of SAE training or dataset composition alone; it is a layer-depth phenomenon traceable to the internal computation of the model.

5.5 Qualitative Feature Analysis and Behaviour Control

This section presents concrete examples of extracted labelled features, source attribution patterns, and behaviour control through the interactive web interface. This pipeline is more than just aggregate statistics.

5.5.1 Feature Case Studies

Table 8 presents four Layer 12 features spanning different semantic categories. For each, the MaxAct contexts show the medical domain in which the feature fires, VocabProj shows what the feature promotes in the output vocabulary, and the source breakdown reveals which training corpus drives the activation.

Table 8: Case studies of four Layer 12 features across distinct medical domains.

Index	Label	Category	Entropy	MaxAct contexts	VocabProj (top 4)	Source
5225	Calcium metabolism	Biochem./Mol.	2.32	Hypercalcemia, hyperparathyroidism, calcitriol, vitamin D	calcium, Bone, bone, Ca	medmcqa 50%, pubmed_qa 50%
1813	Survey methodology	Research Meth.	2.00	Questionnaire design, immunisation surveys, patient sampling	survey, questionnaire, Survey, surveys	pubmed_abs 100%
5124	Hormonal conditions	Clinical/Diag.	3.32	Cushing's syndrome, ACTH, adrenal cortisol, dexamethasone	hormone, hormonal, adrenal, hormones	medmcqa 100%
7951	Spinal conditions	Anatomical	2.00	Disc herniation, lumbar compression, Pott's spine, cauda equina	spinal, spine, discs, disc	medmcqa 75%, pubmed_abs 25%

The calcium metabolism feature (5225) is a perfect example of a well-formed biomedical feature: its VocabProj tokens (calcium, bone, Ca) align precisely with its MaxAct activation pattern, and its balanced source breakdown (50% MedMCQA, 50% PubMed QA) indicates that the concept appears across both clinical and research contexts. By contrast, the survey methodology feature (1813) draws 100% of its activations from PubMed Abstracts, with VocabProj promoting *survey*, *questionnaire*, *surveyed*. This is well a writing-convention feature rather than a medical-knowledge feature. The hormonal conditions feature (5124) is sourced entirely from MedMCQA, consistent with clinical QA content, while the spinal conditions feature (7951) draws primarily from MedMCQA (75%) with some PubMed abstract contribution (25%).

5.5.2 VocabProj Reliability

How important VocabProj is, will depend strongly on the feature quality. For alive monosemantic features at Layer 12, VocabProj produces interpretable output-centric signals: the calcium metabolism feature promotes *calcium*, *bone*, *Ca*; the survey methodology feature promotes *survey*, *questionnaire*. But for dead or highly polysemantic features, VocabProj degenerates into gibberish. Incoherent subword fragments (for example *vern*, *tint*, *.swing*) even if MaxAct is coherent. This pattern can be expected, "bad" feature have no meaningful decoder direction, so projecting it into the unembedding matrix produces random tokens.

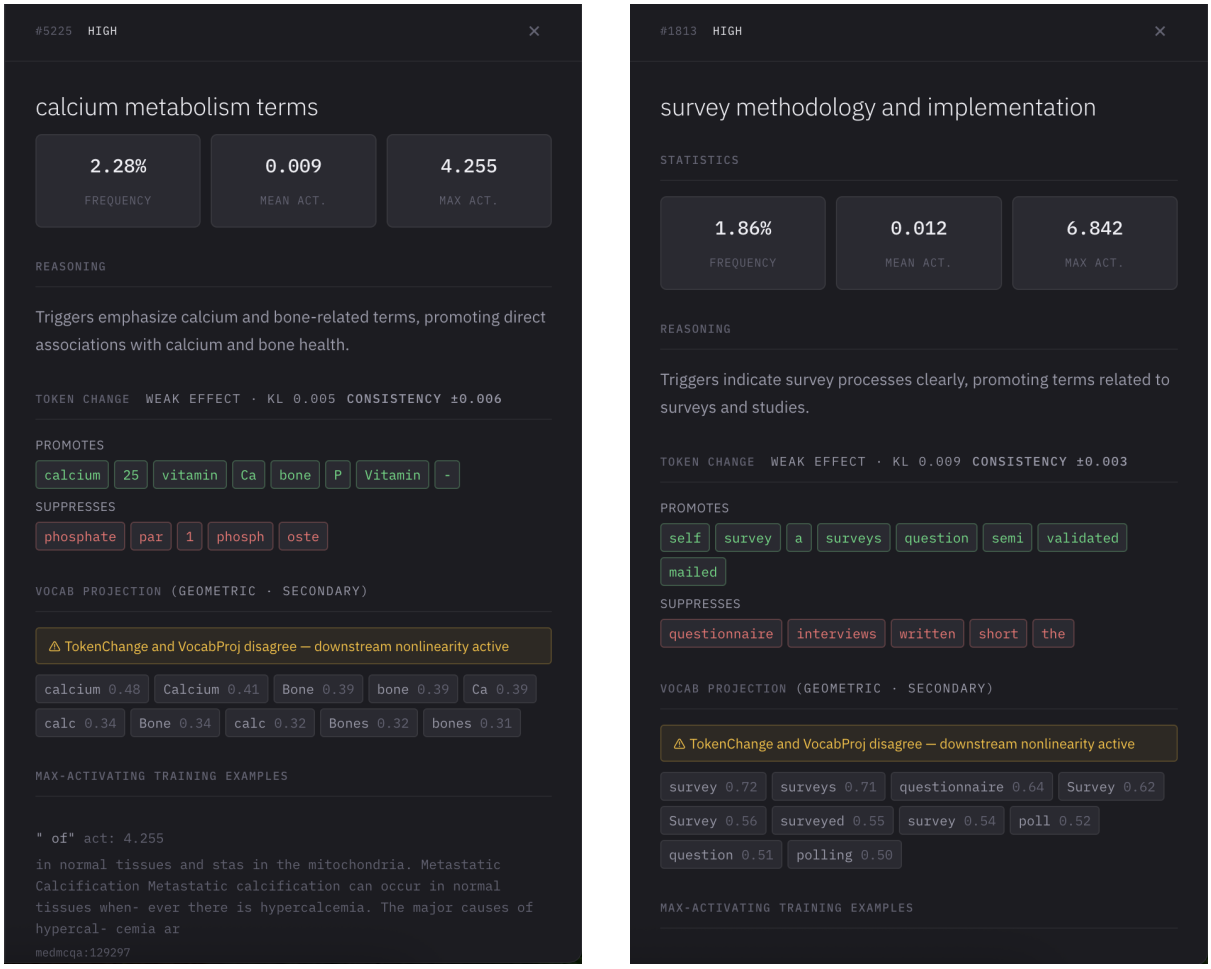


Figure 12: UI feature browser detail views for Feature 5225 (calcium metabolism on the left) and Feature 1813 (survey methodology, right), with all their metadata (MaxAct, VocabProj, source breakdown and quality

5.5.3 Behaviour Control via Feature Steering

The interactive web interface provides a direct way to demonstrate behaviour control. With steering, when a labelled feature is amplified during generation, the output of the model will shift in that direction. The UI also has a baseline generation (no intervention) alongside the steered to comparison.

Screenshots of the behaviour control interface are shown in figures 14, 15, and 16 (Appendix F).

These behaviour control demonstrations prove that extracted features are not just mere statistical artefacts of the SAE, but correspond causally active directions of the residual stream of a model. They influence output generation in a way we can predict and even choose.

6 Discussion

6.1 What SAEs Learn in Specialised Language Models

When training in the mid-layer, the main finding was that the SAE trained on a medical instruction-tuned model predominantly learnt research methodology language rather than actual clinical knowledge. This has important implications for future deployment of interpretability methods in high-stakes domains. The result is consistent: an LLM trained on a specific domain corpus will develop strong and greater internal representations of the linguistics conventions of said domain, rather than the underlying factual content.

Crucially, this finding does not imply that the model lacks medical knowledge. Rather, it suggests that medical knowledge in this model is encoded in the deeper layers, in a sparser, more localised form. The implication for interpretability research is that layer selection is a non-trivial methodological choice: an analysis conducted exclusively at mid-layer residual streams may systematically underestimate the specificity of a model's domain representations. This is a nuanced finding that the existing SAE literature does not address, since most published SAE work uses general-domain models on general web corpora, where the distinction between 'language about a domain' and 'domain knowledge' is less pronounced (Shu et al., 2025). The practical implication of this for safety-critical systems is clear. Interpretation audits on a medical AI which uses mid-level SAE features to describe model behaviour might fail to take into account Layer 12 features which are used in decision-making. In this way, the choice of which layers to interpret should be made based on empirical data, not convenience, in order to avoid missing critical information about model behaviour.

6.2 Cross-Layer Feature Composition

The cross-layer cosine similarity analysis reveals a strongly directional picture of feature evolution. The directional asymmetry in the following table tells us there are two pretty distinct transition types: L4→L8 transition is expansive, whereas the L8→L12 transition is primarily consolidative, with Layer 12 absorbing some Layer 8 directions but generating the majority of

itself.

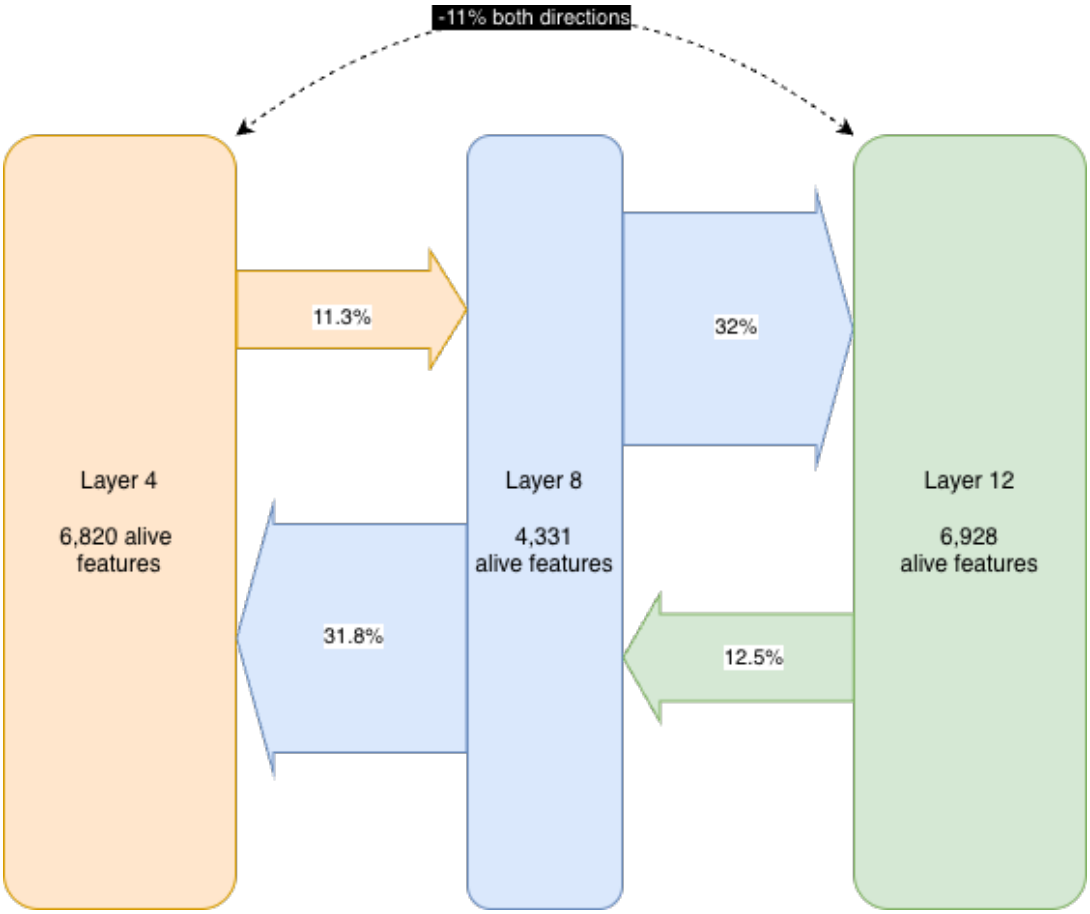


Figure 13: Diagram of the similarity between directional features for Layers 4, 8, and 12. Arrow width corresponds to the percentage which corresponds to directional features from the source layer having cos-similarity larger than 0.7 in the target layer.

In contrast to previous works, our current findings demonstrate that there exists more cross-layer overlap than was observed before. Both mid and late layers are not independent in our case.

6.3 Training Dynamics: Dead Features and the L1–TopK Trade-off

L1 failed to achieve the target sparsity across a wide range of values too many dead neurons. The root cause, as found through training logs, is that the L1 penalty operates on activation magnitude rather than activation probabilities. A feature that activates rarely but with high magnitude contributes a large L1 penalty and is pushed toward zero, while a feature that activates frequently but weakly may escape suppression.

However, with using TopK, we can consider sparsity as an absolute condition. Despite that, we still face a problem of dead features, but it is layer-dependent and sparsity is guaranteed. Most importantly, dead features under the TopK do not affect the alive ones. With L1 the penalty fights both alive and dead features equally. To lower the amount of dead features in Layer 8, we may require either a larger dictionary, more training data, or a more targeted intervention such as layer-specific auxiliary loss weighting or dead feature resampling.

6.4 Limitations

First of all, there is the problem of the dead features. Even though this problem has been greatly reduced in layers 4 and 12, in Layer 8 the issue persists.

Secondly, explainable variance for all three layers is also relatively low, ranging from 41% to 47%. Literature suggests higher explainable variance at around 70-85% for similar models (Bussmann et al., 2024; Gao et al., 2024). The reason for this difference could be the following: Firstly, the 4x ratio we use is relatively small when compared with other experiments. Secondly, the domain-specific medical corpus might contain activations that are difficult to decompose into sparse features. Finally, computational limitations led us to train the model only up to 46 million tokens in 8 epochs, while published benchmarks train their SAEs on hundreds of millions of tokens. Therefore, the current SAE represents a subset of model representations. Despite the limited coverage, however, our main results based on comparing proportions of categories in layers 8 and 12 still stand, as both layers were trained under the same conditions, and we compare proportional distributions rather than absolute feature counts. In order to narrow the gap, the most straightforward method would be increasing the expansion ratio to 8x or 16x and expanding the dataset further.

Third, VocabProj gives an unreliable representation of the dead and polysemantic features in terms of output-centric signals, producing incoherent subword fragments. However, for the alive monosemantic features in Layer 12, VocabProj is coherent. In contrast, TokenChange, which measures the causal effect of a feature's injected decoder direction and is implemented in the pipeline, provides a better output-centric signal. Results of TokenChange are calculated for each feature, however, they were not considered in the semantic analysis of this work and

further exploration of their reliability is planned for future works.

Fourth, the study analyses only the residual stream (`resid_post`) hook points without taking into account attention heads' and MLP sublayer outputs. Training SAEs separately on the latter hook points will give a better understanding of model architecture and its computations (Bricken et al., 2023).

Fifth, using GPT-4o-mini as the labelling model creates a risk of circularity, whereby one LLM analyses the internal representation created by another language model and there may exist systematic bias in the way that certain domain-specific terms are encoded within the model. An evaluation of some labels done by a medical professional should be carried out before the results of this study are used in a clinical setting.

Finally, even though the medical domain was the chosen setting in which to carry out the experiments, the results cannot be said to be domain-specific. It is expected that SAEs will learn stylistic features faster than actual domain knowledge across any corpus that has a clearly defined style like scientific papers, legal documents, computer source code, and other such specialised text collections. The entropy-based monosemanticity indicator and the heuristic pipeline are also not domain-specific.

7 Future Work

The pipeline and findings presented in this thesis open several concrete directions for further research, grouped below by their relationship to the current work's limitations.

7.1 Resolving the Mid-Layer Dead Feature Problem

The first technical issue of concern is the dead feature problem in the middle layer. Several solutions need exploration. The first is the inclusion of layer-wise auxiliary loss weighting, whereby the current loss weight coefficient is uniformly applied to all layers ($\alpha = 1/32$), but the complex activation space in the middle layer may warrant a different approach such as an increased loss weight coefficient or even a completely different auxiliary schedule. The second is the gradual growth of the dictionary through the initial use of a reduced hidden size and subsequent incremental growth as the number of dead features increases. Finally, the newly released JumpReLU activation function (Rajamanoharan et al., 2024) by DeepMind effectively solves the problem of dead features because it uses a learnable jump per feature, obviating the need for a fixed TopK filter.

7.2 Systematic TokenChange Evaluation

TokenChange has been fully implemented within the pipeline and records the causal attributions of each feature, yet it did not form the crux of our semantic evaluation in this paper. A comprehensive comparison between the performance of TokenChange, VocabProj, and MaxAct agreement on a per-layer and per-feature basis will reveal the conditions under which these methods perform reliably and those where they fail. Specifically, the fact that VocabProj shows coherence for live monosemantic features in Layer 12 yet fails to do so in dead features indicates the presence of a threshold above which these methods become trustworthy, and the exact value of this threshold needs to be determined.

7.3 Scaling to Larger Models

The present study is constrained to analysing a 1B-parameter model. Perhaps in a bigger model, our main finding will change, and may occur at different depths. An extension of this method to larger models in the 7B-70B parameter range will reveal if the occurrence of this transition always happens at a depth of one quarter through the layers, or whether it will occur at a shallower depth with deeper models. The computational expense is the major challenge here.

7.4 Additional Hook Points

The analysis is restricted to the residual stream (`resid_post`). Training SAEs separately on attention head outputs and MLP sublayer activations would decompose the model's computation more finely, revealing which components contribute the methodology-convention features observed at Layer 8 and which contribute the domain-knowledge features that emerge at Layer 12. Transcoders (Shu et al., 2025), which map between activation spaces at different layers rather than compressing a single layer, offer another route: they could directly characterise the computation that transforms Layer 8's writing-convention representations into Layer 12's domain-knowledge representations, providing a mechanistic account of the transition this thesis documents descriptively.

7.5 Domain Transfer

The empirical observation found is namely that SAEs acquire genre knowledge before domain-specific knowledge. This will hold across other disciplines apart from medicine. Corpora in the legal domain can be expected to yield the development of features related to legal language conventions and legal doctrines at the mid-level and deep-level stages respectively because of its unique patterns of citation and formulaic language use. In the coding domain, features related to coding syntax, indentation, matching brackets for example, they are also likely to be acquired prior to features related to computational concepts like recursion or sorting algorithms.

7.6 Human Expert Validation

Labelling process uses an LLM (GPT-4o-mini) to generate semantic labels, leading to a possible circularity. A specific validation study where a healthcare practitioner examines a representative sample of labelled features, ranging from highly confident to less confident within each semantic class, will estimate the percentage of false positives and false negatives in the automated labelling process. This is a necessary step before any implementation of the labelling process within the interpretability audit.

8 Conclusion

In this research project, there was a precise question to be addressed, which was, ”Do the features discovered by sparse autoencoders from a domain-specific language model contain domain knowledge or merely the linguistic conventions used in domain-specific text?” Through empirical studies conducted using Llama 3.2 1B Instruct trained on 46 million tokens of biomedical text, the results revealed that it contains both, just at different depths. This hypothesis is therefore partially confirmed: mid-layer features capture writing conventions mostly, but deeper layers do encode substantive domain knowledge.

The pipeline trained BatchTopK SAE independently for Layers 4, 8, and 12 of the residual stream of the model. Over 3,300 unique features were obtained from just Layer 12, and additionally, the source attribution method was used to identify which document gave rise to a certain feature.

First, at mid-layer depth (Layer 8), the most prominent category of features reflects conventional linguistic expressions used in research methodology (hedging claims, reporting of statistics, method section language) rather than actual biomedical content. At Layer 12, on the contrary, categories relating to domain knowledge (Clinical/Diagnostic, Pharmacology, Anatomy, Biochemical/Molecular) increased, whereas features related to research methodology diminished.

Second, feature source attribution shows this result in an alternative way, not relying on the semantic labelling task. Research Methodology features are based mostly on PubMed abstract text, whereas Clinical/Diagnostic features rely more on the MedMCQA text collection in the Q&A format. Corpus provenance is obtained through an analysis of raw token origins to avoid dependency on LLM performance in labelling tasks.

Third is the study of cross-layer cosine similarities which shows that the layers are nearly independent (about 88%–90% of the features per layer do not resemble those in other layers). But this independence is not symmetric since Layer 8 is largely based on Layer 4, while Layer 12 mostly creates features itself.

8.1 Contributions

The individual contributions of this thesis are:

1. An end-to-end pipeline for the interpretation of domain-specific language models using BatchTopK SAE training, multiple rounds of feature extraction (MaxAct, VocabProj, TokenChange), entropy-driven evaluation, automatic label generation in two stages, source identification on a per-feature basis, and layer-by-layer comparison. The entire pipeline and code are open-source.

2. Per-feature source identification: a technique for tracking the training dataset sources of activation examples of each feature to provide a quantitative provenance assessment for each identified feature.

3. Layer-depth dependence of transitioning from writing conventions to domain knowledge and its implications for the selection of layers in interpretability tasks.

8.2 Implications for Practice

An interpretability audit, of a domain specific AI system that focuses exclusively on mid-layer SAE features can be misled into understanding how a domain writes rather than what it knows. In safety-sensitive applications like healthcare, law, and finance, the difference between learning to write and learning to reason is crucial. The results obtained here imply that depth-stratification analysis is an essential aspect of applied interpretability studies, and that per-feature source attribution is an integral step in such audits.

It may appear that the findings and the methodology are specific to the biomedical corpus at hand; they are not. Any distinct writing style found in science papers, legal documentation, code bases, etc., will yield similar depth-stratification: SAEs will learn genre rules on a shallower layer and learn the domain-specific content on a deeper one. Both the entropy-based proxy for monosemanticity and the quality heuristic, as well as source attribution method itself, are domain-independent.

A Hyperparameter Configuration

Tables 9–12 list every hyperparameter used in the final run. All values are taken directly from the source code (`config.py` and `main.py`) ("Github Capstone Project", n.d.).

Table 9: Model and dataset configuration.

Parameter	Value
Base model	`meta-llama/Llama-3.2-1B-Instruct`
Model hidden dimension (d_{model})	2,048
Target layers	4, 8, 12
Hook point	`resid_post`
Training samples	500,000 (453,000 after filtering)
Token count	46.3M
Data sources	MedMCQA, PubMed QA, PubMed Abstracts
Input formatting	Llama 3.2 chat template

Table 10: SAE architecture configuration.

Parameter	Value
Architecture	BatchTopK (Bussmann et al. 2024)
Expansion factor	$4\times$ ($d_{\text{hidden}} = 8{,}192$)
Top-K	64 features per token
Auxiliary loss coefficient (α)	$1/32$
Auxiliary K	256
Encoder initialisation	Decoder$^\top$ (Gao et al. 2024)
Bias initialisation (`b_pre`)	Mean of data sample
Decoder normalisation	Unit-norm columns after each step
Activation (training)	Batch-level top-K
Activation (inference)	Per-sample top-K

Table 11: Training configuration.

Parameter	Value
Optimiser	Adam
Learning rate	1×10^{-4}
LR schedule	Cosine annealing to 10% of initial
Batch size	4,096 tokens
Epochs	8
Loss function	MSE (reconstruction) + auxiliary loss

Table 12: Feature analysis and labelling configuration.

Parameter	Value
MaxAct examples per feature	20
Max context length (tokens)	128
VocabProj top tokens	20
VocabProj mean-centring	Enabled
VocabProj chunk size	512 features
Cross-layer similarity threshold	0.7 (cosine)
Labelling model	GPT-4o-mini
Labelling batch size	50 features

B Quality Score Details

Not every feature extracted is labelled; they must first pass through a quality quality score $Q(j) \in [0,1]$ designed to identify reliably monosemantic candidates(for labelling). The score is defined as:

$$Q(j) = \frac{1}{7}\left(2 \cdot S_{\text{freq}}(j) + S_{\text{cov}}(j) + S_{\text{vocab}}(j) + S_{\text{peak}}(j) + S_{\text{div}}(j) + S_{\text{ent}}(j)\right),$$

where the seven weighted components are:

- $S_{\text{freq}}(j)$ **(double-weighted)**: Activation frequency score. Any features outside the range $[10^{-4}, 0.30]$ is excluded before scoring. Within that range though, S_{freq} is maximised near 0.02 (2% firing rate), falling off symmetrically on either side. We give frequency a double weight because as empirically found out, it is the strongest predictor for monosemanticity.

- $S_{\text{cov}}(j)$: MaxAct coverage score: $\min(|\text{MaxAct}_j|, 10)/10$, where $|\text{MaxAct}_j|$ is the number of MaxAct tokens collected for a feature j. The max is 10, so features with above 10 score 1.0. The more the better.

- $S_{\text{vocab}}(j)$: VocabProj coverage score: $\min(|\text{VocabProj}_j|, 10)/10$, where $|\text{VocabProj}_j|$ is the number of vocabulary tokens returned for feature j. Similarly to the MaxAct coverage. The fuller the vocabulary projection the better.

- $S_{\text{peak}}(j)$: Mean activation score: $\min(\bar{h}_j/10, 1.0)$, where $\bar{h}_j$ is the mean activation of

feature j conditioned on firing. Features with higher mean activations tend to be more distinctive and are rewarded up to 10.

- $S_{\text{div}}(j)$: Token surface-form diversity: the number of distinct token strings among MaxAct examples, normalised. This prevents a feature from scoring highly if all its MaxAct examples are the same surface form despite occurring in varied contexts.

- $S_{\text{ent}}(j)$: Normalised entropy score: $1 - H(j)/\log_2(M)$, where M is the number of MaxAct examples. Lower entropy (more monosemantic) is better.

C Semantic Category Taxonomy

All labelled features are assigned to one of eleven broad categories by a rule-based classifier implemented in `label_features.py` in the source code(“Github Capstone Project”, n.d.). Table 13 lists each category with its description and keywords from the source code.

Table 13: Eleven-category semantic taxonomy. The *Keywords* column shows a representative subset of the matching terms used by the rule-based classifier; the full keyword lists are in the source code.

Category	Description	Representative Keywords
Structural/Linguistic	Positional features, discourse markers, formatting tokens, syntactic patterns, citation and reference patterns, early/late sequence position features.	structural, positional, punctuation, digit, conjunction, pronoun, citation, abbreviation, list marker
Research Methodology	Experimental design terms, statistical reporting conventions, research evaluation vocabulary, quantifiable measurement language.	research, trial, randomized, cohort, meta-analysis, statistical, p-value, confidence interval, regression, cross-sectional
Pharmacology	Drug names, dosage language, administration routes, therapeutic drug levels, pharmaceutical approval terminology.	drug, medication, therapeutic, dose, antibiotic, inhibitor, chemotherap, analgesic, pharmacokinetic, contraindic
Microbiology/Infectious	Viral and bacterial infections, disease transmission patterns, pathogen interaction descriptions.	bacteri, viral, virus, infect, pathogen, sepsis, biofilm, gram-positive, gram-negative
Oncology	Tumour grading and classification, cancer prevalence, biopsy and sampling terminology.	cancer, tumor, oncol, malign, metasta, carcinoma, lymphoma, staging, biopsy, remission
Mental Health/Psych	Psychiatric disorder terminology, psychological stress factors, cognitive process descriptions.	psych, depress, anxiety, cognitive, schizoph, bipolar, dementia, counseling, mood
Biochemical/Molecular	Cellular structure, molecular biology terminology, genetic and biochemical pathways, immune signalling.	gene, protein, enzyme, pathway, receptor, hormone, amino acid, dna, rna, mitochond, ion channel
Epidemiological	Patient demographics, population risk factors, public health initiatives, incidence and prevalence terminology.	epidemiol, prevalence, incidence, mortality, public health, outbreak, demographic, pandemic, screening
Anatomical	Body part names, anatomical origins of conditions, organ system vocabulary, physiological descriptions.	anatomy, organ, tissue, muscle, bone, nerve, artery, ligament, cortex, cerebr, thorac, abdomin
Clinical/Diagnostic	Patient-facing clinical language, symptom descriptions, syndrome characterisations, diagnostic criteria, risk assessment, surgical terminology.	clinical, diagnosis, symptom, syndrome, surgical, imaging, mri, prognosis, diabetes, hypertens, cardiac, respiratory, renal
General/Other	All remaining features too specific or niche for the above categories (e.g. chemical reaction processes, intravenous solution preparations, biological mechanisms).	Fallback: assigned when no keyword rule matches.

D Selected Feature Examples

Table 14 presents 20 high-confidence features from Layer 12, that cover all 11 semantic categories. Source abbreviations: MC = MedMCQA, PA = PubMed Abstracts, PQ = PubMedQA. QS = quality score, H = MaxAct entropy (bits).

Table 14: 20 high-confidence SAE features from Layer 12

Category	Feature Label	QS	H	Top MaxAct	Top VocabProj	Source (%)
Clinical/ Diagnostic	risk assessment parameters	0.71	3.32	`hesion, rate, risk`	`by, significantly, considerably`	MC 20, PQ 80
	supplement-related discussions	0.71	3.32	`in, ine, acid`	`supplementation, supplements, intervention`	PA 20, PQ 80
Research Methodology	systematic reviews methodology	0.70	3.17	`trial, review, of`	`meta, Meta, Meta`	MC 11, PA 33, PQ 56
	statistical significance descriptors	0.69	3.17	`showed, a, statistically`	`statistically, Significant, significant`	PA 11, PQ 89
Structural/ Linguistic	publication reporting conventions	0.67	2.32	`of, summarize, summarizes`	`recent, current, latest`	PA 80, PQ 20
	context-end / late-position†	0.65	3.32	`-specific, ase, our`	`};, );, ));`	MC 100
Biochemical/ Molecular	acid-base balance discussions	0.71	3.17	`or, (, At`	`pH, acid, acidity`	MC 90, PQ 10
	cellular response processes	0.70	3.17	`of, stimulates, stimulating`	`proliferation, apoptosis, production`	MC 22, PQ 78
Pharmacology	therapeutic drug levels	0.72	3.12	`ime, imus, peak`	`levels, Levels, level`	MC 60, PQ 40

Continued on next page

Category	Feature Label	QS	*H*	Top MaxAct	Top VocabProj	Source (%)
	FDA pharmaceutical approval	0.71	3.32	`been, the, FDA`	`FDA, drug, approval`	MC 20, PA 50, PQ 30
Anatomical	anatomical origins of conditions	0.71	3.32	`elial, muscular, ermal`	`ones, nature, Ones`	MC 80, PQ 20
General/ Other	chemical reaction processes	0.71	3.32	`a, zed, reversible`	`reactions, reaction, Reaction`	MC 90, PA 10
	timing and initiation of effects	0.71	3.32	`begins, initiated, obtained`	`within, Within, immediately`	MC 100
Epidemiological	public health interventions	0.69	3.32	`and, at, ;`	`malaria, rural, poverty`	MC 90, PQ 10
Microbiology/ Infectious	viral interaction descriptions	0.68	2.00	`viral, the, V`	`virus, viral, viruses`	PQ 100
	fungal infection identifiers	0.65	2.00	`., by, against`	`fungal, fungus, fungi`	MC 75, PA 25
Oncology	tumor grading classification	0.72	2.95	`as, C, 1`	`grade, grades, Grade`	MC 40, PA 30, PQ 30
	cancer mutation terminology	0.69	3.32	`of, carrying, oring`	`mutations, mutation, Mut`	MC 30, PQ 70

Continued on next page

Category	Feature Label	QS	*H*	Top MaxAct	Top VocabProj	Source (%)
Mental Health/Psych	psychiatric disorder terminology	0.68	3.32	`bid, existing, many`	`psychiatric, mental, Mental`	MC 20, PQ 80
	behavioral reinforcement concepts	0.60	3.32	`through, be, are`	`rewards, reward, Reward`	MC 80, PA 10, PQ 10

† Heuristic-positional label; all others labelled by LLM.

E Layer 4 and Layer 8 Category Distributions

Table 15: Layer 4 semantic category distribution

Category	Count	% of Labelled
Structural/Linguistic	708	35.7%
General/Other	558	28.2%
Clinical/Diagnostic	296	14.9%
Research Methodology	92	4.6%
Biochemical/Molecular	88	4.4%
Pharmacology	88	4.4%
Anatomical	63	3.2%
Epidemiological	34	1.7%
Microbiology/Infectious	24	1.2%
Mental Health/Psych	18	0.9%
Oncology	13	0.7%
Total	**1,982**	**100%**

Table 16: Layer 8 semantic category distribution

Category	Count	% of Labelled
Structural/Linguistic	626	31.8%
General/Other	552	28.0%
Clinical/Diagnostic	337	17.1%
Research Methodology	103	5.2%
Pharmacology	94	4.8%
Biochemical/Molecular	91	4.6%
Anatomical	69	3.5%
Epidemiological	37	1.9%
Oncology	22	1.1%
Microbiology/Infectious	20	1.0%
Mental Health/Psych	19	1.0%
Total	**1,970**	**100%**

F Behaviour Control UI Screenshots

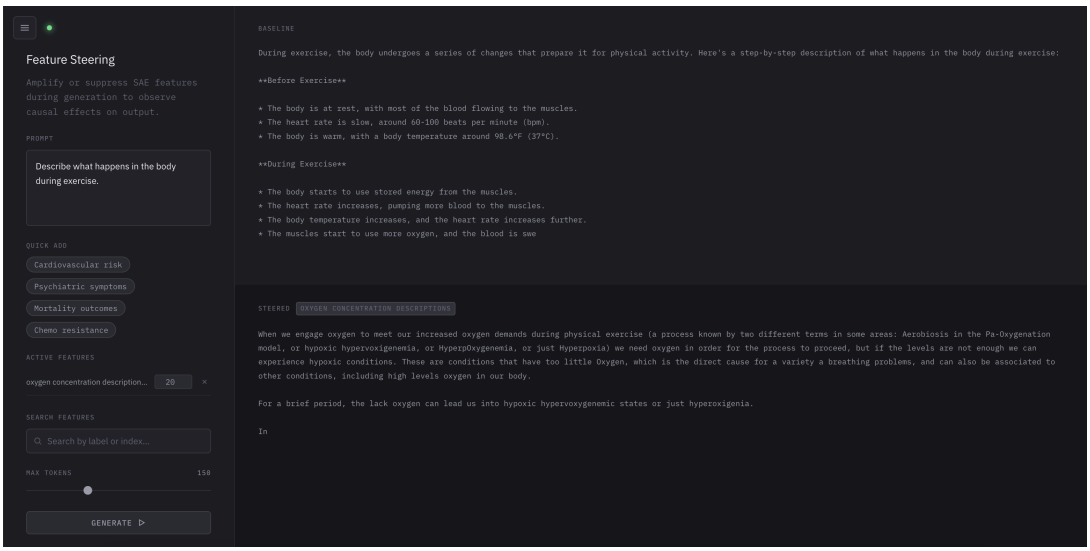


Figure 14: The following screenshot shows generation steering. The steered output shows a clear emphasis on the chosen feature: oxygen concentration descriptions

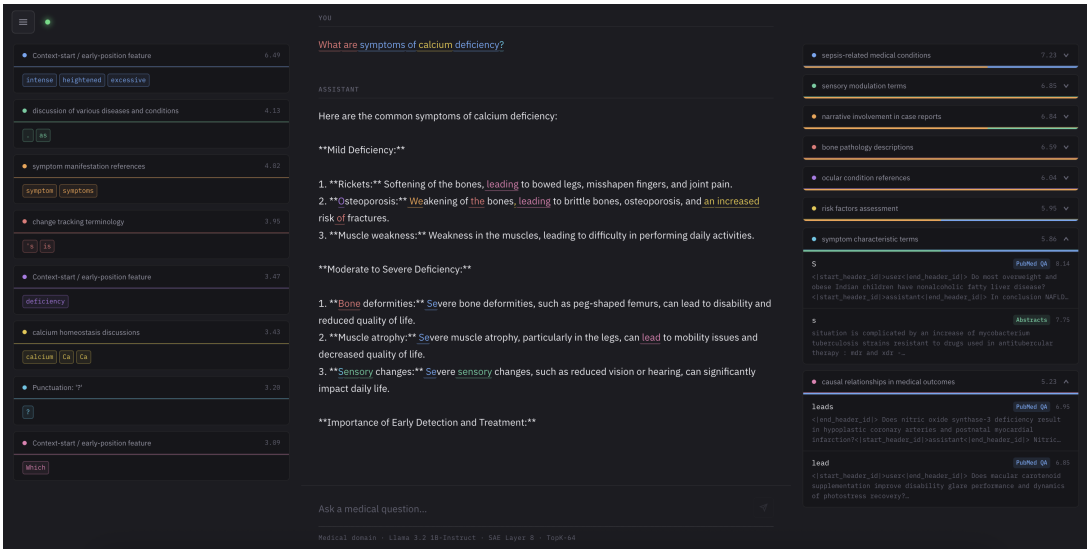


Figure 15: Chat interface showing the resulting model output, with on the left panel the features activated by the input, and on the right panel the features that promoted the output.

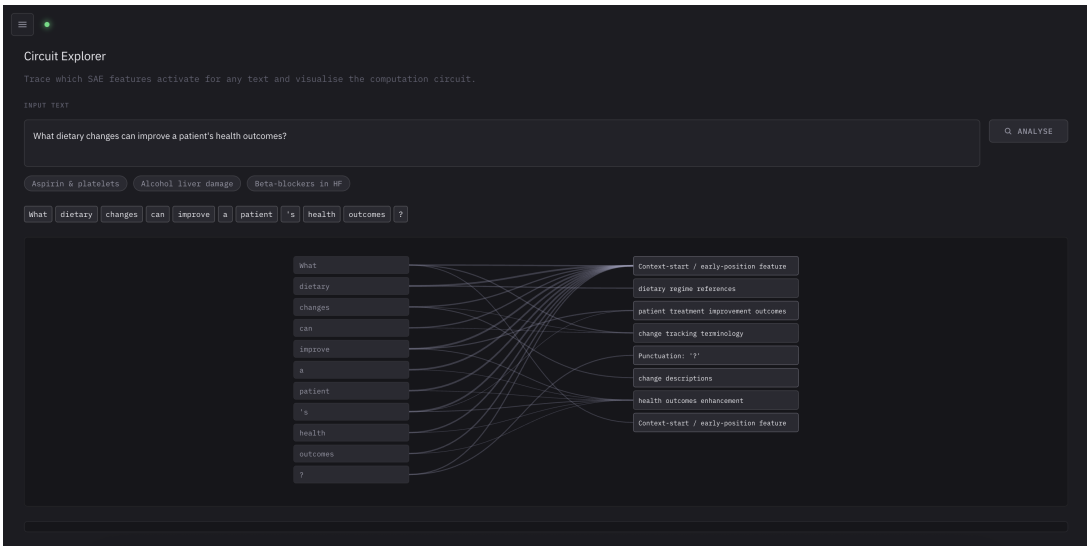


Figure 16: Circuit explorer view. The screenshot shows a D3.js Sankey diagram mapping input tokens to activated features.

References

Bricken, T., Templeton, A., Batson, J., Chen, B., Jermyn, A., Conerly, T., Turner, N., Anil, C., Denison, C., Askell, A., Lasenby, R., Wu, Y., Kravec, S., Schiefer, N., Maxwell, T., Joseph, N., Hatfield-Dodds, Z., Tamkin, A., Nguyen, K., … Olah, C. (2023). Towards monosemanticity: Decomposing language models with dictionary learning [https://transformer-circuits.pub/2023/monosemantic-features/index.html]. *Transformer Circuits Thread*.

Bussmann, B., Leask, P., & Nanda, N. (2024). Batchtopk sparse autoencoders. https://arxiv.org/abs/2412.06410

Chen, H., Vondrick, C., & Mao, C. (2024). Selfie: Self-interpretation of large language model embeddings. https://arxiv.org/abs/2403.10949

Cohan, A., Dernoncourt, F., Kim, D. S., Bui, T., Kim, S., Chang, W., & Goharian, N. (2018). A discourse-aware attention model for abstractive summarization of long documents. *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, 615–621. https://doi.org/10.18653/v1/N18-2097

Cunningham, H., Ewart, A., Riggs, L., Huben, R., & Sharkey, L. (2023). Sparse autoencoders find highly interpretable features in language models. https://arxiv.org/abs/2309.08600

Dunefsky, J., Chlenski, P., & Nanda, N. (2024). Transcoders find interpretable llm feature circuits. https://arxiv.org/abs/2406.11944

Gao, L., la Tour, T. D., Tillman, H., Goh, G., Troll, R., Radford, A., Sutskever, I., Leike, J., & Wu, J. (2024). Scaling and evaluating sparse autoencoders. https://arxiv.org/abs/2406.04093

Github capstone project. (n.d.). https://github.com/BaptisteEtroy/Capstone

Gokaslan, A., & Cohen, V. (2019). Openwebtext corpus.

Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., Yang, A., Fan, A., Goyal, A., Hartshorn, A., Yang, A., Mitra, A., Sravankumar, A., Korenev, A., Hinsvark, A., … Ma, Z. (2024). The llama 3 herd of models. https://arxiv.org/abs/2407.21783

Gur-Arieh, Y., Mayan, R., Agassy, C., Geiger, A., & Geva, M. (2025). Enhancing automated interpretability with output-centric feature descriptions. https://arxiv.org/abs/2501.08319

Heap, T., Lawson, T., Farnik, L., & Aitchison, L. (2026). Automated interpretability metrics do not distinguish trained and random transformers. https://arxiv.org/abs/2501.17727

Hugging face llama 3.2 1b instruct documentation. (n.d.). https://huggingface.co/meta-llama/Llama-3.2-1B-Instruct

Hugging face stream documentation. (n.d.). https://huggingface.co/docs/datasets/en/stream

Jin, Q., Dhingra, B., Liu, Z., Cohen, W., & Lu, X. (2019). Pubmedqa: A dataset for biomedical research question answering. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2567–2577.

Kingma, D. P., & Ba, J. (2017). Adam: A method for stochastic optimization. https://arxiv.org/abs/1412.6980

Laptev, D., Balagansky, N., Aksenov, Y., & Gavrilov, D. (2025). Analyze feature flow to enhance interpretation and steering in language models. https://arxiv.org/abs/2502.03032

Marks, S., Rager, C., Michaud, E. J., Belinkov, Y., Bau, D., & Mueller, A. (2025). Sparse feature circuits: Discovering and editing interpretable causal graphs in language models. https://arxiv.org/abs/2403.19647

Nanda, N., & Bloom, J. (2022). Transformerlens.

OpenAI. (2024). GPT-4o mini: Advancing cost-efficient intelligence. https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/

Pal, A., Umapathi, L. K., & Sankarasubbu, M. (2022). Medmcqa: A large-scale multi-subject multi-choice dataset for medical domain question answering. In G. Flores, G. H. Chen, T. Pollard, J. C. Ho, & T. Naumann (Eds.), *Proceedings of the conference on health, inference, and learning* (pp. 248–260, Vol. 174). PMLR. https://proceedings.mlr.press/v174/pal22a.html

Paulo, G., Mallen, A., Juang, C., & Belrose, N. (2025). Automatically interpreting millions of features in large language models. https://arxiv.org/abs/2410.13928

Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). Language models are unsupervised multitask learners. https://openai.com/blog/better-language-models/

Rajamanoharan, S., Lieberum, T., Sonnerat, N., Conmy, A., Varma, V., Kramár, J., & Nanda, N. (2024). Jumping ahead: Improving reconstruction fidelity with jumprelu sparse autoencoders. https://arxiv.org/abs/2407.14435

Shu, D., Wu, X., Zhao, H., Rai, D., Yao, Z., Liu, N., & Du, M. (2025). A survey on sparse autoencoders: Interpreting the internal mechanisms of large language models. https://arxiv.org/abs/2503.05613

Templeton, A., Conerly, T., Marcus, J., Lindsey, J., Bricken, T., Chen, B., Pearce, A., Citro, C., Ameisen, E., Jones, A., Cunningham, H., Turner, N. L., McDougall, C., MacDiarmid, M., Freeman, C. D., Sumers, T. R., Rees, E., Batson, J., Jermyn, A., ... Henighan, T. (2024). Scaling monosemanticity: Extracting interpretable features from claude 3 sonnet. *Transformer Circuits Thread*. https://transformer-circuits.pub/2024/scaling-monosemanticity/index.html

Yu, M., Zhou, Z., Aloqaily, M., Wang, K., Huang, B., Wang, S., Jin, Y., & Wen, Q. (2025). Backdoor attribution: Elucidating and controlling backdoor in language models. https://arxiv.org/abs/2509.21761